K-modes clustering

What is K-Modes Clustering?

KModes is a clustering algorithm used in data science to group similar data points into clusters based on their categorical attributes. Unlike traditional clustering algorithms that use distance metrics, KModes works by identifying the modes or most frequent values within each cluster to determine its centroid. KModes is ideal for clustering categorical data such as customer demographics, market segments, or survey responses. It is a powerful tool for data analysts and scientists to gain insights into their data and make informed decisions.

KModes vs KMeans

KMeans uses mathematical measures (distance) to cluster continuous data. The lesser the distance, the more similar our data points are. Centroids are updated by Means. But for categorical data points, we cannot calculate the distance. So we go for KModes algorithm. It uses the dissimilarities(total mismatches) between the data points. The lesser the dissimilarities the more similar our data points are. It uses Modes instead of means.

How Does the KModes Algorithm Work?

Unlike Hierarchical clustering methods, we need to upfront specify the K.

1. Pick K observations at random and use them as leaders/clusters
2. Calculate the dissimilarities and assign each observation to its closest cluster

3. Define new modes for the clusters
4. Repeat 2–3 steps until there are is no re-assignment required

I hope you got the basic idea of the KModes algorithm by now. So let us quickly take an example to illustrate the working step by step.

Example: Imagine we have a dataset that has the information about hair color, eye color, and skin color of persons. We aim to group them based on the available information(maybe we want to suggest some styling ideas)

Hair color, eye color, and skin color are all categorical variables. Below 👇 is how our dataset looks like.

person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Image of our data

Alright, we have the sample data now. Let us proceed by defining the number of clusters(K)=3

Step 1: Pick K observations at random and use them as leaders/clusters

I am choosing P1, P7, P8 as leaders/clusters

Leaders			
P1	blonde	amber	fair
P7	red	green	fair
P8	black	hazel	fair

person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Leaders and Observations

Step 2: Calculate the dissimilarities(no. of mismatches) and assign each observation to its closest cluster

Iteratively compare the cluster data points to each of the observations. Similar data points give 0, dissimilar data points give 1.

Leaders			
P1	blonde	amber	fair
P7	red	green	fair
P8	black	hazel	fair

person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Comparing leader/Cluster P1 to the observation P1 gives 0 dissimilarities.

Leaders			
P1	blonde	amber	fair
P7	red	green	fair
P8	black	hazel	fair

person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Comparing leader/cluster P1 to the observation P2 gives 3(1+1+1) dissimilarities.

Likewise, calculate all the dissimilarities and put them in a matrix as shown below and assign the observations to their closest cluster(cluster that has the least dissimilarity)

	Cluster 1 (P1)	Cluster 2 (P7)	Cluster 3 (P8)	Cluster
P1	0 ✓	2	2	Cluster 1
P2	3 ✓	3	3	Cluster 1
P3	3	1 ✓	3	Cluster 2
P4	3	3	1 ✓	Cluster 3
P5	1 ✓	2	2	Cluster 1
P6	3	3	2 ✓	Cluster 3
P7	2	0 ✓	2	Cluster 2
P8	2	2	0 ✓	Cluster 3

Dissimilarity matrix (Image by Author)

After step 2, the observations P1, P2, P5 are assigned to cluster 1; P3, P7 are assigned to Cluster 2; and P4, P6, P8 are assigned to cluster 3.

Note: If all the clusters have the same dissimilarity with an observation, assign to any cluster randomly. In our case, the observation P2 has 3 dissimilarities with all the leaders. I randomly assigned it to Cluster 1.

Step 3: Define new modes for the clusters

Mode is simply the **most observed value**.

Mark the observations according to the cluster they belong to. Observations of Cluster 1 are marked in Yellow, Cluster 2 are marked in Brick red, and Cluster 3 are marked in Purple.

person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Looking for Modes (Image by author)

Considering one cluster at a time, for each feature, look for the Mode and update the new leaders.

Explanation: Cluster 1 observations(P1, P2, P5) has brunette as the most observed hair color, amber as the most observed eye color, and fair as the most observed skin color.

Note: If you observe the same occurrence of values, take the mode randomly. In our case, the observations of Cluster 3(P3, P7) have one occurrence of brown, fair skin color. I randomly chose brown as the mode.

Below are our new leaders after the update.

New Leaders			
	hair color	eye color	skin color
Cluster 1	brunette	amber	fair
Cluster 2	red	green	fair
Cluster 3	black	hazel	brown

Obtained new leaders

Repeat steps 2–4

After obtaining the new leaders, again calculate the dissimilarities between the observations and the newly obtained leaders.

New Leaders			
	hair color	eye color	skin color
Cluster 1	brunette	amber	fair
Cluster 2	red	green	fair
Cluster 3	black	hazel	brown

person	hair color	eye color	skin color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Comparing Cluster 1 to the observation P1 gives 1 dissimilarity.

New Leaders			
	hair color	**eye color**	**skin color**
Cluster 1	brunette	amber	fair
Cluster 2	red	green	fair
Cluster 3	black	hazel	brown
person	**hair color**	**eye color**	**skin color**
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Comparing Cluster 1 to the observation P2 gives 2 dissimilarities.

Likewise, calculate all the dissimilarities and put them in a matrix. Assign each observation to its closest cluster.

	Cluster 1	Cluster 2	Cluster 3	Cluster
P1	1 ✓	2	3	Cluster 1
P2	2 ✓	3	2	Cluster 1
P3	3	1 ✓	2	Cluster 2
P4	3	3	0 ✓	Cluster 3
P5	0 ✓	2	3	Cluster 1
P6	3	3	1 ✓	Cluster 3
P7	2	0 ✓	3	Cluster 2
P8	2	2	1 ✓	Cluster 3

The observations P1, P2, P5 are assigned to Cluster 1; P3, P7 are assigned to Cluster 2; and P4, P6, P8 are assigned to Cluster 3.

We stop here as we see there is no change in the assignment of observations.

Implementation of KModes in Python

Begin with Importing necessary libraries

```
# importing necessary libraries

import pandas as pd

import numpy as np

# !pip install kmodes

from kmodes.kmodes import KModes

import matplotlib.pyplot as plt

%matplotlib inline
```

1. Creating Toy Dataset

Python Code:

```
import pandas as pd
```

```
import numpy as np


# Create toy dataset

hair_color = np.array(['blonde', 'brunette', 'red', 'black', 'brunette', 'black', 'red', 'black'])

eye_color = np.array(['amber', 'gray', 'green', 'hazel', 'amber', 'gray', 'green', 'hazel'])

skin_color = np.array(['fair', 'brown', 'brown', 'brown', 'fair', 'brown', 'fair', 'fair'])

person = ['P1','P2','P3','P4','P5','P6','P7','P8']

data = pd.DataFrame({'person':person, 'hair_color':hair_color, 'eye_color':eye_color,
'skin_color':skin_color})

data = data.set_index('person')

print(data)
```

person	hair_color	eye_color	skin_color
P1	blonde	amber	fair
P2	brunette	gray	brown
P3	red	green	brown
P4	black	hazel	brown
P5	brunette	amber	fair
P6	black	gray	brown
P7	red	green	fair
P8	black	hazel	fair

Image of our dataset

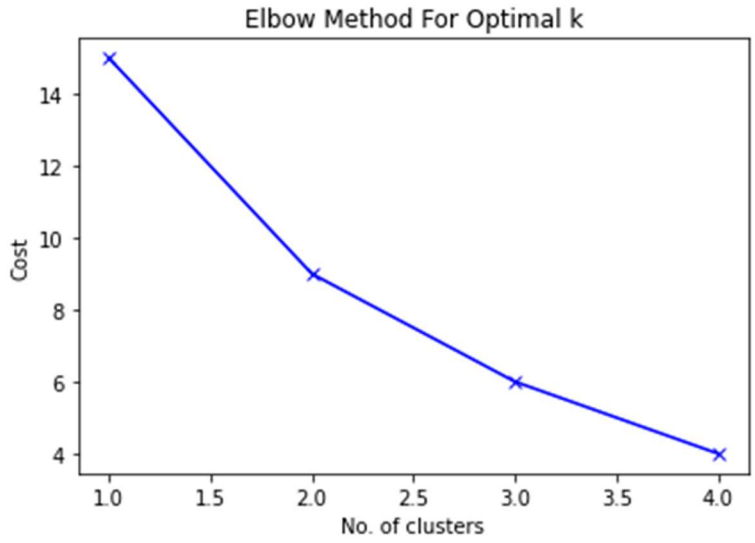
2. Scree Plot or Elbow Curve to Find Optimal Kvalue

For KModes, plot cost for a range of K values. Cost is the sum of all the dissimilarities between the clusters.

Select the K where you observe an elbow-like bend with a lesser cost value.

```
# Elbow curve to find optimal K
cost = []
K = range(1,5)
for num_clusters in list(K):
    kmode = KModes(n_clusters=num_clusters, init = "random", n_init = 5, verbose=1)
    kmode.fit_predict(data)
    cost.append(kmode.cost_)


plt.plot(K, cost, 'bx-')
plt.xlabel('No. of clusters')
plt.ylabel('Cost')
plt.title('Elbow Method For Optimal k')
plt.show()
```



Elbow Curve

We can see a bend at K=3 in the above graph indicating 3is the optimal number of clusters.

3. Build a Model with 3 Clusters

```
# Building the model with 3 clusters

kmode = KModes(n_clusters=3, init = "random", n_init = 5, verbose=1)

clusters = kmode.fit_predict(data)

clustersCopy Code
```

Finally, insert the predicted cluster values in our original dataset.

```
data.insert(0, "Cluster", clusters, True)

data
```

:

person	Cluster	hair_color	eye_color	skin_color
P1	0	blonde	amber	fair
P2	0	brunette	gray	brown
P3	2	red	green	brown
P4	1	black	hazel	brown
P5	0	brunette	amber	fair
P6	1	black	gray	brown
P7	2	red	green	fair
P8	1	black	hazel	fair

Dataset after inserting predicted cluster values

Inference from the model predictions: P1, P2, P5 are merged as a cluster; P3, P7 are merged; and P4, P6, P8 are merged.

K-Modes clustering is an unsupervised machine learning technique. It is a partition clustering algorithm used to group a dataset into K clusters. K-Modes clustering can be used in machine learning applications that need to partition data having categorical variables.

K-Modes clustering is an iterative algorithm that starts by selecting k initial data points as centroids of the cluster. After that, each data point in the dataset is assigned to a cluster based on its similarity with the centroids. After creating clusters for the first time, we select a new centroid in each cluster using the mod of each feature in the data. After selecting new clusters, we calculate their dissimilarity from each data point and regroup the clusters. This process

continues until the process converges and there is no change to the clusters in two consecutive iterations.

The K-Modes clustering partitions the data into two mutually exclusive groups. Hence, it is termed a partitioning clustering algorithm.

Why Cannot We Use K-means Clustering on Categorical Data?

To calculate the distance or dissimilarity between two data points in the dataset, the K-means clustering algorithm uses euclidean distance or Manhattan distance. Similarly, the k-means algorithm uses the mean of the features of the data points in a cluster to calculate centroids for new clusters. I have discussed the entire process of k-means clustering with a numerical example. If required, you can go through this article.

We are able to calculate the Euclidean distance, Manhattan distance, or mean of the features in a cluster because K-means clustering is primarily used for numeric data. These operations cannot be defined for a dataset having categorical features.

Even if you annotate the data while data preprocessing, there is not much meaning to the assigned numerical labels in the dataset if there is no order. Hence, we cannot use k-means clustering on a dataset with categorical features. Instead, we use the K-Modes clustering algorithm.

Calculation of Dissimilarity Score Between Two Data Points in K-Modes Clustering

In K-Modes clustering, we cannot use Euclidean or Manhattan distance as a dissimilarity measure for the data points. Hence, we will use a different measure as discussed here. For calculating the dissimilarity measure, consider the following dataset.

Self-organizing map:

The Self Organizing Map is one of the most popular neural models. It belongs to the category of the competitive learning network. The SOM is based on unsupervised learning, which means that is no human intervention is needed during the training and those little needs to be known about characterized by the input data. We could, for example, use the SOM for clustering membership of the input data. The SOM can be used to detect features inherent to the problem and thus has also been called SOFM the Self Origination Feature Map.

The Self Organized Map was developed by professor kohenen which is used in many applications.

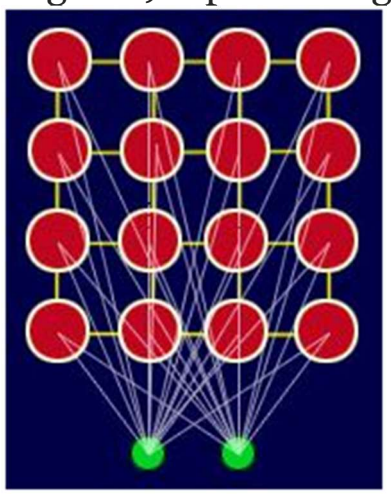
the purpose of SOM is that it's providing a data visualization technique that helps to understand high dimensional data by reducing the dimension of data to map. SOM also represents the clustering concept by grouping similar data together.

Therefore it can be said that Self Organizing Map reduces data dimension and displays similarly among data.

Self Organization Maps Network Architecture?

For the purposes, we'll be discussing a two-dimensional SOM. The network is created from a 2D lattice of 'nodes', each of which is fully connected to the input layer. The below Figure shows a very small

Kohonen network of 4 X 4 nodes connected to the input layer (shown in green) representing a two-dimensional vector.



A simple Kohonen network

Each node has a specific topological position (an x, y coordinate in the lattice) and contains a vector of weights of the same dimension as the input vectors. That is to say, if the training data consists of vectors, *V*, of *n* dimensions:

V1, V2, V3…Vn

Then each node will contain a corresponding weight vector *W,* of *n* dimensions:

W1, W2, W3…Wn

The lines connecting the nodes in the above Figure are only there to represent adjacency and do not signify a connection as normally

indicated when discussing a neural network. There are no lateral connections between nodes within the lattice.

4. How Self Organization Maps work?

4.1: Learning Algorithm Overview

A SOM does not need a target output to be specified unlike many other types of network. Instead, where the node weights match the input vector, that area of the lattice is selectively optimized to more closely resemble the data for the class the input vector is a member of. From an initial distribution of random weights, and over many iterations, the SOM eventually settles into a map of stable zones. Each zone is effectively a feature classifier, so you can think of the graphical output as a type of feature map of the input space.

Training occurs in several steps and over many iterations:

1. Each node's weights are initialized.

2. A vector is chosen at random from the set of training data and presented to the lattice.

3. Every node is examined to calculate which ones weights are most like the input vector. The winning node is commonly known as the Best Matching Unit (BMU).

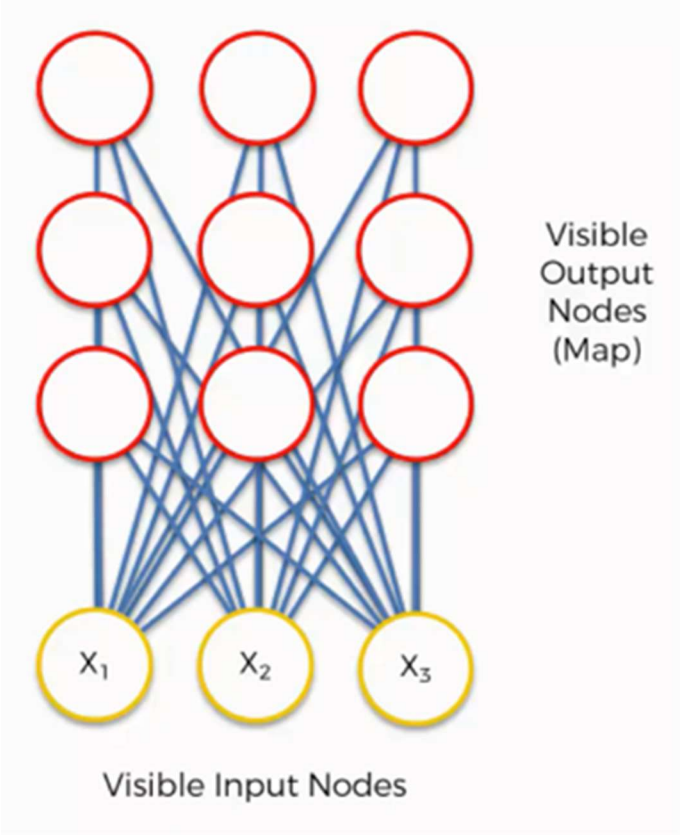
4. The radius of the neighborhood of the BMU is now calculated. This is a value that starts large, typically set to the 'radius' of the lattice, but diminishes each time-step. Any nodes found within this radius are deemed to be inside the BMU's neighborhood.

5. Each neighboring node's (the nodes found in step 4) weights are adjusted to make them more like the input vector. The closer a node is to the BMU; the more its weights get altered.

6. Repeat step 2 for N iterations.

4.2: Learning Algorithm in Details.

Now it's time for us to learn how SOMs learn. Are you ready? Let's begin. Right here we have a very basic self-organizing map.



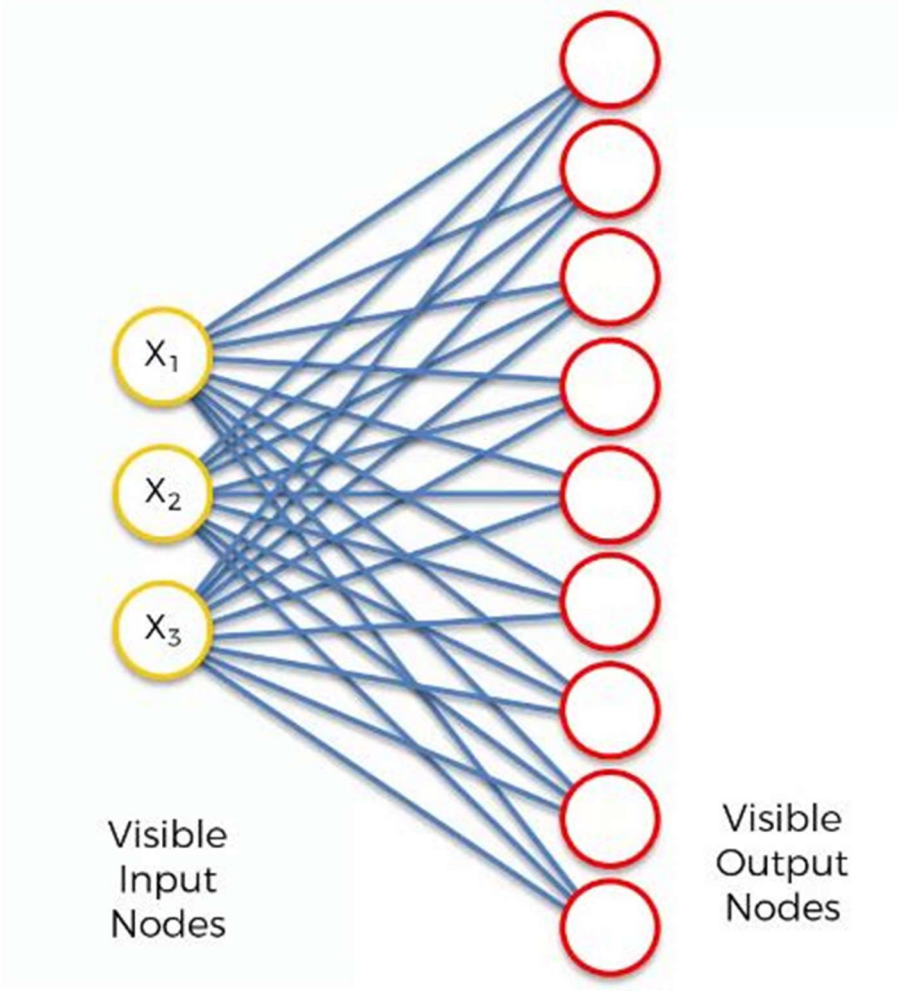
Our input vectors amount to three features, and we have nine output nodes.

That being said, it might confuse you to see how this example shows three input nodes producing nine output nodes. Don't get puzzled by that. The three input nodes represent three columns (dimensions) in the dataset, but each of these columns can contain thousands of rows. The output nodes in a SOM are always two-dimensional.

Now what we'll do is turn this SOM into an input set that would be more familiar to you from when we discussed the supervised machine

learning methods (artificial, convolutional, and recurrent neural networks) in earlier chapters.

Consider the Structure of Self Organizing which has 3 visible input nodes and 9 outputs that are connected directly to input as shown below fig.



Our input nodes values are:

$X_1 = 0.7$

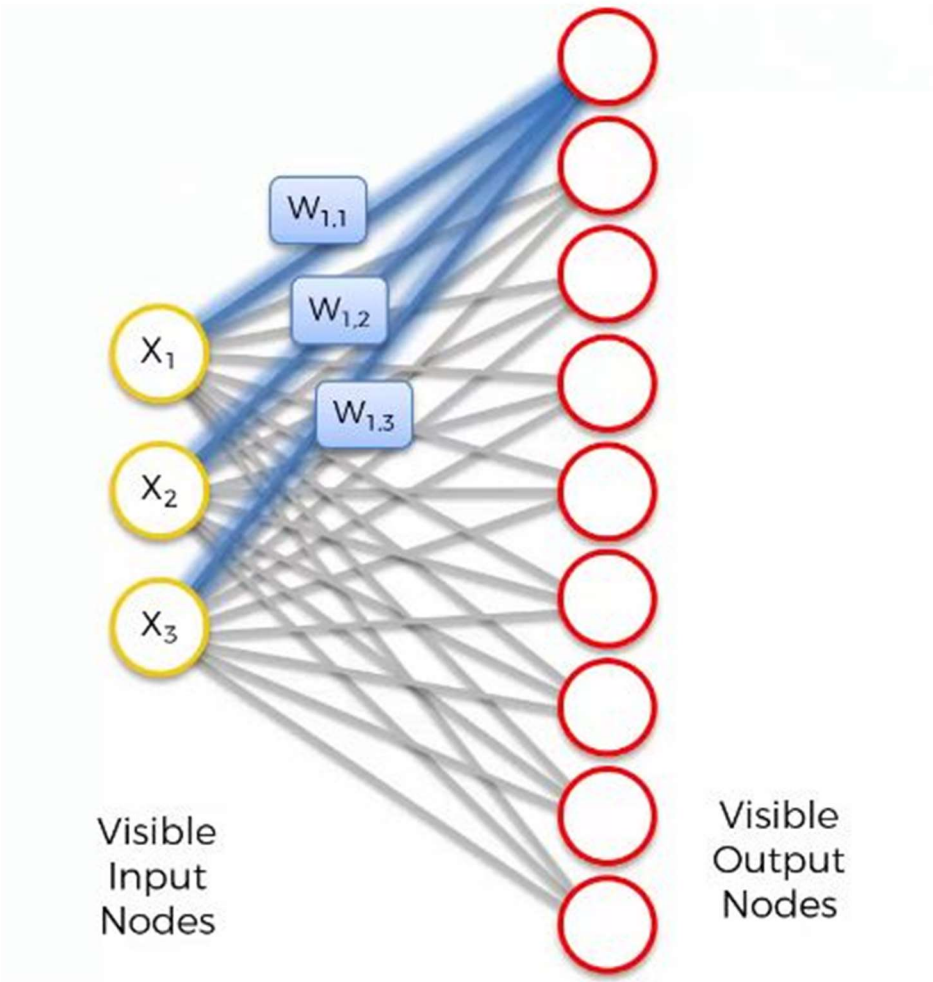
$X_2 = 0.6$

$X_3 = 0.9$

Now let's take a look at each step in detail.

Step 1: Initializing the Weights

Now, let's take the topmost output node and focus on its connections with the input nodes. As you can see, there is a weight assigned to each of these connections.



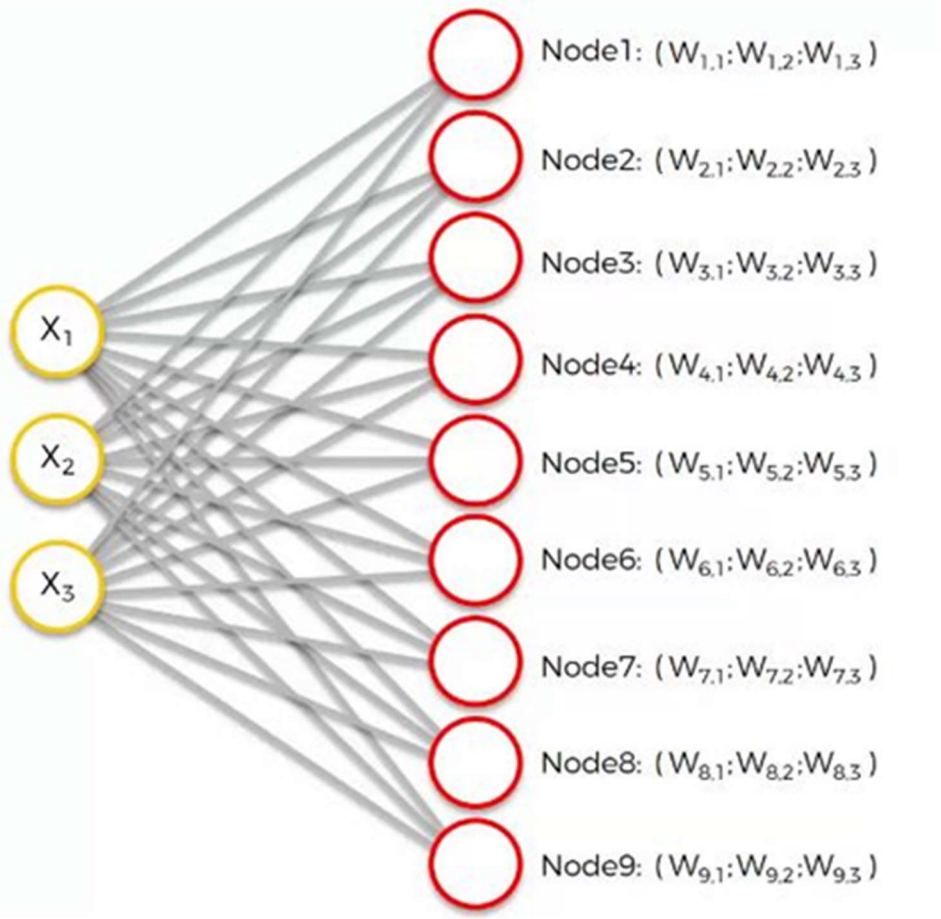
Again, the word “weight” here carries a whole other meaning than it did with artificial and convolutional neural networks. For instance, with artificial neural networks we multiplied the input node’s value by the weight and, finally, applied an activation function. With SOMs, on the other hand, there is no activation function.

Weights are not separate from the nodes here. In a SOM, the weights belong to the output node itself. Instead of being the result of adding up the weights, the output node in a SOM contains the weights as its

coordinates. Carrying these weights, it sneakily tries to find its way into the input space.

In this example, we have a 3D dataset, and each of the input nodes represents an x-coordinate. The SOM would compress these into a single output node that carries three weights. If we happen to deal with a 20-dimensional dataset, the output node, in this case, would carry 20 weight coordinates.

Each of these output nodes does not exactly become parts of the input space, but try to integrate into it nevertheless, developing imaginary places for themselves.



We have randomly initialized the values of the weights (close to 0 but not 0).

$W_{1,1} = 0.31$	$W_{1,2} = 0.22$	$W_{1,3} = 0.10$
$W_{2,1} = 0.21$	$W_{2,2} = 0.34$	$W_{2,3} = 0.19$
$W_{3,1} = 0.39$	$W_{3,2} = 0.42$	$W_{3,3} = 0.45$
$W_{4,1} = 0.25$	$W_{4,2} = 0.32$	$W_{4,3} = 0.62$
$W_{5,1} = 0.24$	$W_{5,2} = 0.31$	$W_{5,3} = 0.16$
$W_{6,1} = 0.52$	$W_{6,2} = 0.33$	$W_{6,3} = 0.42$
$W_{7,1} = 0.31$	$W_{7,2} = 0.22$	$W_{7,3} = 0.10$
$W_{8,1} = 0.12$	$W_{8,2} = 0.41$	$W_{8,3} = 0.19$
$W_{9,1} = 0.34$	$W_{9,2} = 0.40$	$W_{9,3} = 0.51$

Step 2: Calculating the Best Matching Unit

The next step is to go through our dataset. For each of the rows in our dataset, we'll try to find the node closest to it.

Say we take row number 1, and we extract its value for each of the three columns we have. We'll then want to find which of our output nodes is closest to that row.

To determine the best matching unit, one method is to iterate through all the nodes and calculate the Euclidean distance between each node's

weight vector and the current input vector. The node with a weight vector closest to the input vector is tagged as the BMU.

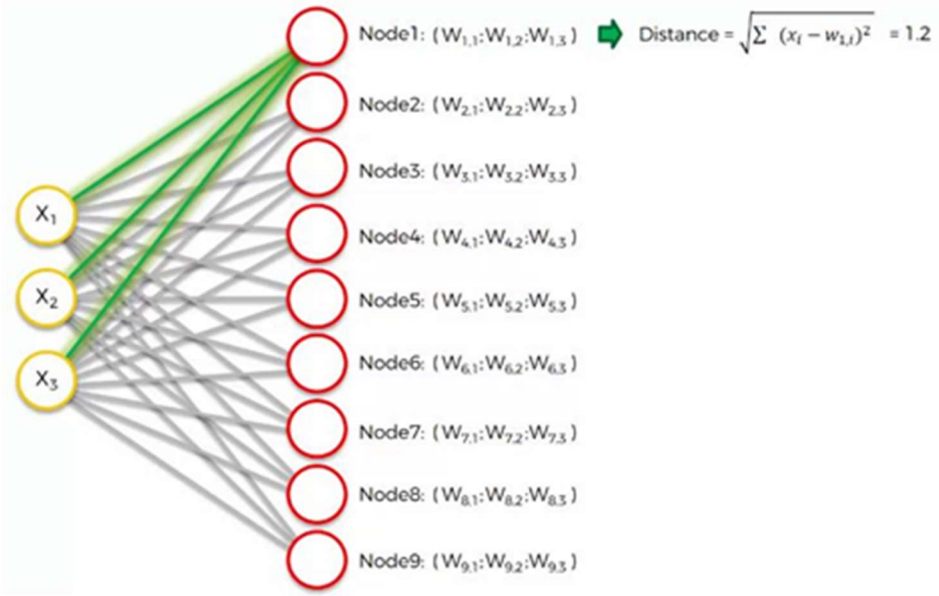
The Euclidean distance is given as:

$$\text{Distance} = \sqrt{\sum_{i=0}^{i=n}(X_i - W_i)^2}$$

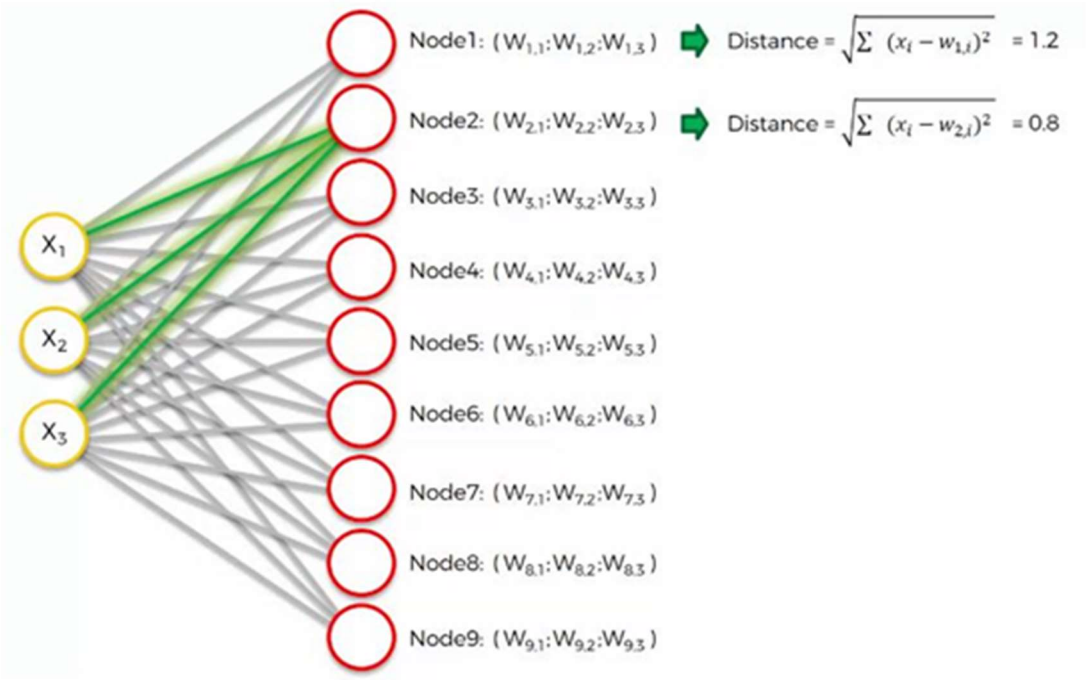
Where X is the current input vector and W is the node's weight vector.

Let's calculate the Best Match Unit using the Distance formula.

For 1st Nodes:

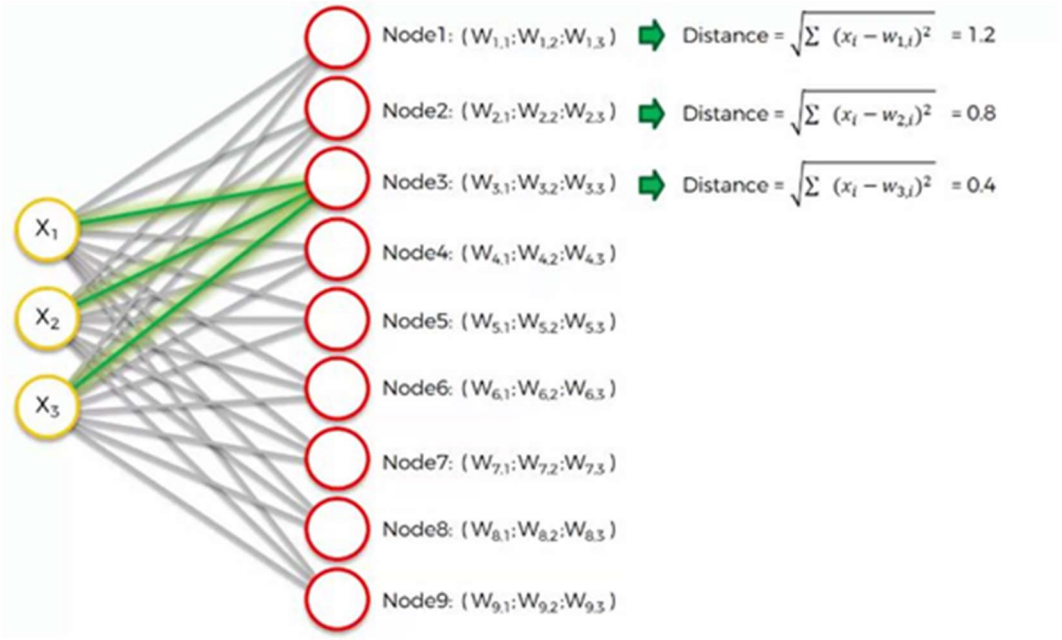


For 2nd Nodes:

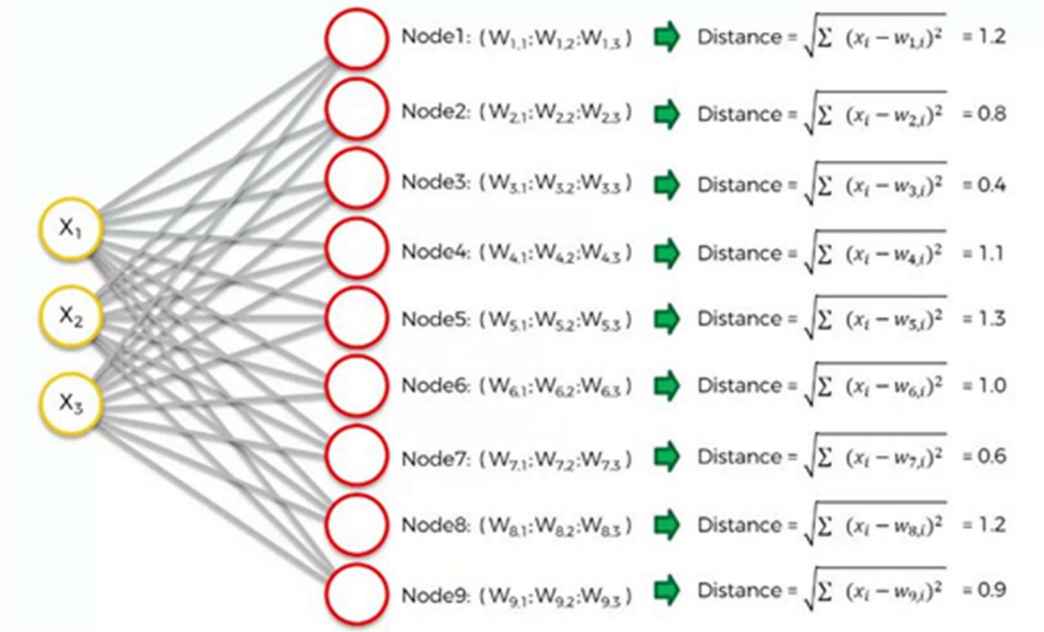


For 3rd Nodes:

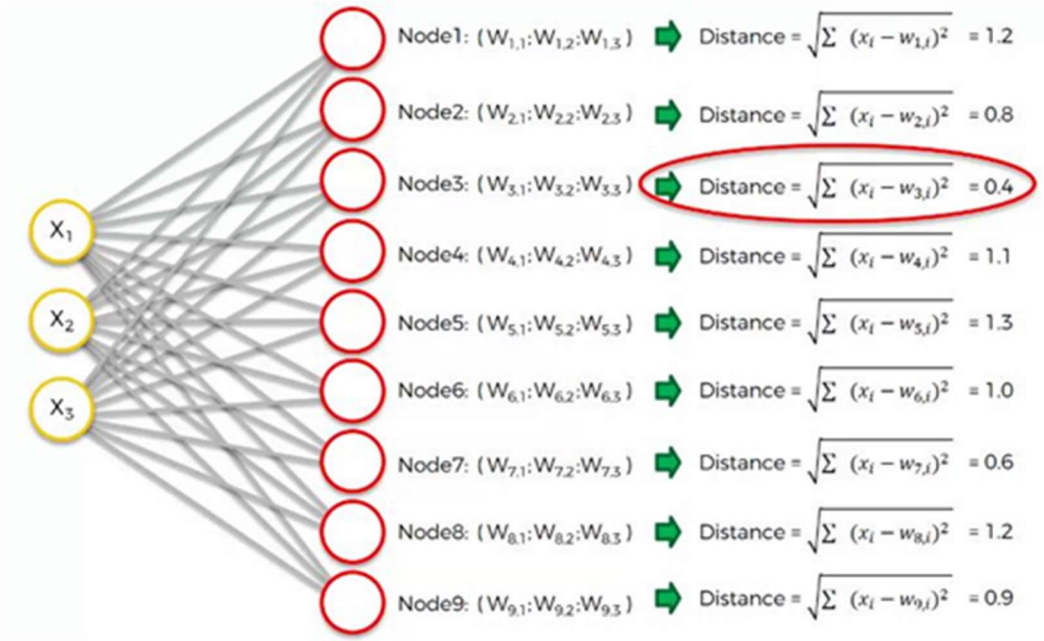
Similarly, way we calculate all remaining Nodes the same way as you can see below.



Similarly, way we calculate all remaining Nodes the same way as you can see below.



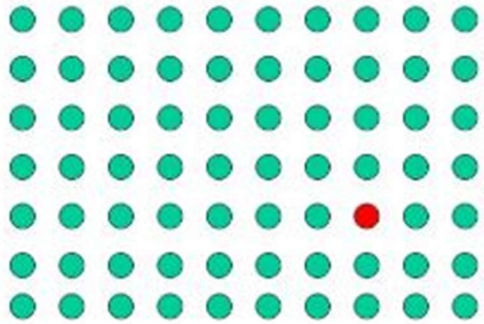
Since we have calculated all the values of respected Nodes. Now it's time to calculate the Best Match Unit.



As we can see, node number 3 is the closest with a distance of 0.4. We will call this node our BMU (best-matching unit).

What happens next?

To understand this next part, we'll need to use a larger SOM.



Supposedly you now understand what the difference is between weights in the SOM context as opposed to the one we were used to when dealing with supervised machine learning.

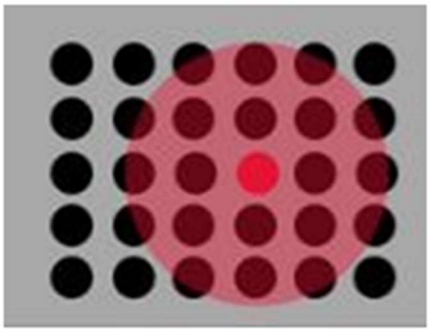
The red circle in the figure above represents this map's BMU. Now, the new SOM will have to update its weights so that it is even closer to our dataset's first row. The reason we need this is that our input nodes cannot be updated, whereas we have control over our output nodes.

In simple terms, our SOM is drawing closer to the data point by stretching the BMU towards it. The end goal is to have our map as aligned with the dataset as we see in the image on the far right

Step 3: Calculating the size of the neighborhood around the BMU

This is where things start to get more interesting! Each iteration, after the BMU has been determined, the next step is to calculate which of the other nodes are within the BMU's neighborhood. All these nodes will have their weight vectors altered in the next step. So how do we do that? Well, it's not too difficult… first, you calculate what the radius of the neighborhood should be and then it's a simple application of good ol' Pythagoras to determine if each node is within the radial distance or not.

The figure shows an example of the size of a typical neighborhood close to the commencement of training.



You can see that the neighborhood shown above is centered around the BMU (red-point) and encompasses most of the other nodes and circle show radius.

The size of the neighborhood around the BMU is decreasing with an exponential decay function. It shrinks on each iteration until reaching just the BMU

$$\sigma(t) = \sigma_0 \exp(-\frac{t}{\lambda})$$

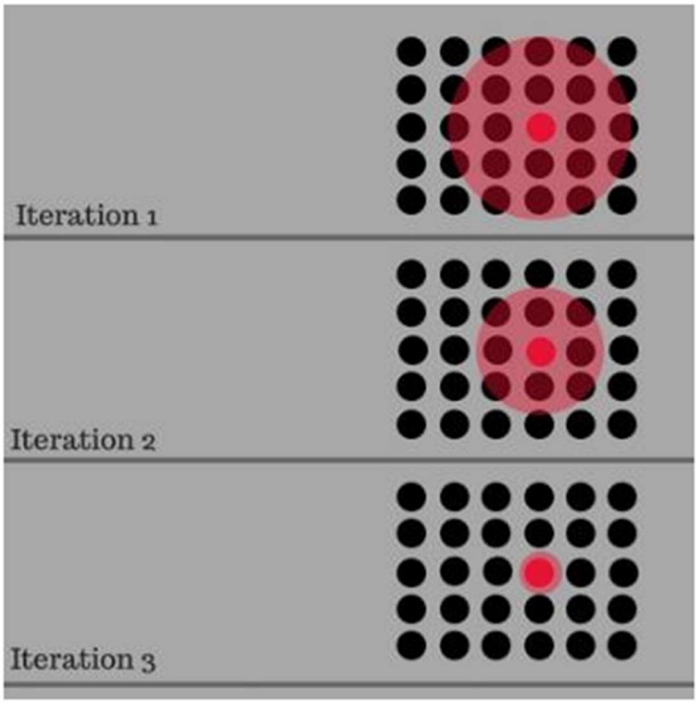
σ_0 = *the width of lattice at time zero*

t = *the current time step*

λ = *the time constant*

Where t = 0, 1, 2, 3….

Figure below shows how the neighborhood decreases over time after each iteration



Over time the neighborhood will shrink to the size of just one node… the BMU.

Now we know the radius, it's a simple matter to iterate through all the nodes in the lattice to determine if they lay within the radius or not. If a node is found to be within the neighborhood then its weight vector is adjusted as follows in Step 4.

How to set the radius value in the self-organizing map?

It depends on the range and scale of your input data. If you are mean-zero standardizing your feature values, then try $\sigma=4$. If you are normalizing feature values to a range of [0, 1] then you can still try $\sigma=4$, but a value of $\sigma=1$ might be better. Remember, you have to decrease the learning rate α and the size of the neighborhood function with increasing iterations, as none of the metrics stay constant throughout the iterations in SOM.

It also depends on how large your SOM is. If it's a 10 by 10, then use for example $\sigma=5$. Otherwise, if it's a 100 by 100 map, use $\sigma=50$.

In unsupervised classification, σ is sometimes based on the Euclidean distance between the centroids of the first and second closest clusters.

Step 4: Adjusting the Weights

Every node within the BMU's neighborhood (including the BMU) has its weight vector adjusted according to the following equation:

New Weights = Old Weights + Learning Rate (Input Vector — Old Weights)

$W(t+1) = W(t) + L(t) (V(t) — W(t))$

Where t represents the time-step and L is a small variable called the learning rate, which decreases with time. What this equation is sayiWhatnewly adjusted weight for the node is equal to the old weight (W), plus a fraction of the difference (L) between the old weight and the input vector (V).

So according to our example are Node 4 is Best Match Unit (as you can see in step 2) corresponding their weights:

$W_{3,1} = 0.39$	$W_{3,2} = 0.42$	$W_{3,3} = 0.45$
Input Vector: $X_1 = 0.7$	$X_2 = 0.6$	$X_3 = 0.9$

Learning rate = 0.5

So update that weight according to the above equation

For W3,1

New Weights = Old Weights + Learning Rate (Input Vector1 — Old Weights)

New Weights = 0.39 + 0.5 (0.7–0.39)

New Weights = 0.545

For W3,2

New Weights = Old Weights + Learning Rate (Input Vector2 — Old Weights)

New Weights = 0.42 + 0.5 (0.6–0.42)

New Weights = 0.51

For W3,3

New Weights = Old Weights + Learning Rate (Input Vector3 — Old Weights)

New Weights = 0.45 + 0.5 (0.9–0.45)

New Weights = 0.675

Updated weights:

$W_{3,1} = 0.545$ $W_{3,2} = 0.51$ $W_{3,3} = 0.67$

So in this way we update the weights.

The decay of the learning rate is calculated each iteration using the following equation:

$$L(t) = L_0 \exp(-\frac{t}{\lambda})$$

As training goes on, the neighborhood gradually shrinks. At the end of the training, the neighborhoods have shrunk to zero sizes.

$$\Theta(t) = \exp\left(-\frac{dist^2}{2\sigma^2(t)}\right)$$

$\Theta(t) =$ *Influence rate*

$\sigma(t) =$ *width of the lattice at time t*

The influence rate shows the amount of influence a node's distance from the BMU has on its learning. In the simplest form influence rate is equal to 1 for all the nodes close to the BMU and zero for others, but a Gaussian function is common too. Finally, from a random distribution of weights and through many iterations, SOM can arrive at a map of stable zones. In the end, interpretation of data is to be done by a human but SOM is a great technique to present the invisible patterns in the data.

Practical Implementation of SOMs?

Fraud Detection

According to a recent report published by Markets & Markets, the Fraud Detection and Prevention Market is going to be worth USD 33.19 Billion by 2021. This is a huge industry and the demand for advanced Deep Learning skills is only going to grow. That's why we have included this case study in this chapter.

The business challenge here is about detecting fraud in credit card applications. We will be creating a Deep Learning model for a bank and given a dataset that contains information on customers applying for an advanced credit card.

This is the data that customers provided when filling the application form. Our task is to detect potential fraud within these applications. That means that by the end of the challenge, we will come up with an explicit list of customers who potentially cheated on their applications.

Dataset

Data Set Information: This file concerns credit card applications. All attribute names and values have been changed to meaningless symbols to protect the confidentiality of the data.
This dataset is interesting because there is a good mix of attributes —

continuous, nominal with small numbers of values, and nominal with larger numbers of values. There are also a few missing values.

Attribute Information: There are 6 numerical and 8 categorical attributes. The labels have been changed for the convenience of the statistical algorithms. For example, attribute 4 originally had 3 labels p,g, gg and these have been changed to labels 1,2,3.
A1: 0,1 CATEGORICAL (formerly: a,b)
A2: continuous.
A3: continuous.
A4: 1,2,3 CATEGORICAL (formerly: p,g,gg)
A5: 1, 2,3,4,5,6,7,8,9,10,11,12,13,14 CATEGORICAL (formerly: ff,d,i,k,j,aa,m,c,w, e, q, r,cc, x)
A6: 1, 2,3, 4,5,6,7,8,9 CATEGORICAL (formerly: ff,dd,j,bb,v,n,o,h,z)
A7: continuous.
A8: 1, 0 CATEGORICAL (formerly: t, f)
A9: 1, 0 CATEGORICAL (formerly: t, f)
A10: continuous.
A11: 1, 0 CATEGORICAL (formerly t, f)
A12: 1, 2, 3 CATEGORICAL (formerly: s, g, p)
A13: continuous.
A14: continuous.
A15: 1,2 class attribute (formerly: +,-)

Dataset sample:

CustomerID	A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	A11	A12	A13	A14	Class
15776156	1	22.08	11.46	2	4	4	1.585	0	0	0	1	2	100	1213	0
15739548	0	22.67	7	2	8	4	0.165	0	0	0	0	2	160	1	0
15662854	0	29.58	1.75	1	4	4	1.25	0	0	0	1	2	280	1	0
15687688	0	21.67	11.5	1	5	3	0	1	1	11	1	2	0	1	1
15715750	1	20.17	8.17	2	6	4	1.96	1	1	14	0	2	60	159	1
15571121	0	15.83	0.585	2	8	8	1.5	1	1	2	0	2	100	1	1
15726466	1	17.42	6.5	2	3	4	0.125	0	0	0	0	2	60	101	0
15660390	0	58.67	4.46	2	11	8	3.04	1	1	6	0	2	43	561	1
15663942	1	27.83	1	1	2	8	3	0	0	0	0	2	176	538	0
15638610	0	55.75	7.08	2	4	8	6.75	1	1	3	1	2	100	51	0
15644446	1	33.5	1.75	2	14	8	4.5	1	1	4	1	2	253	858	1
15585892	1	41.42	5	2	11	8	5	1	1	6	1	2	470	1	1
15609356	1	20.67	1.25	1	8	8	1.375	1	1	3	1	2	140	211	0
15803378	1	34.92	5	2	14	8	7.5	1	1	6	1	2	0	1001	1
15599440	1	58.58	2.71	2	8	4	2.415	0	0	0	1	2	320	1	0
15692408	1	48.08	6.04	2	4	4	0.04	0	0	0	0	2	0	2691	1
15683168	1	29.58	4.5	2	9	4	7.5	1	1	2	1	2	330	1	1

Let's solve the problem

Part 1: Data Preprocessing

1.1 Import the Libraries

In this step, we import three Libraries in Data Preprocessing part. A library is a tool that you can use to make a specific job. First of all, we import the **numpy** library used for multidimensional array then import the **pandas** library used to import the dataset and in last we import **matplotlib** library used for plotting the graph.

```
# import the Libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

1.2 Import the dataset

In this step, we import the dataset to do that we use the **pandas** library. After import our dataset we define our dependent and independent variable. Our independent variables are 1 to 12 attributes as you can see in the sample dataset which we call '**X**' and dependent is our last attribute which we call '**y**' here.

Note: we will build the SOMs model which is unsupervised deep learning so we are working with independent variables.

```
# import the dataset
dataset = pd.read_csv('dataset\Credit_Card_applications.csv')
X = dataset.iloc [: ,:-1].values    # independent variables
y = dataset.iloc [: , -1].values    # dependent variables
```

1.3 Feature Scaling

Feature Scaling is the most important part of data preprocessing. If we see our dataset then some attribute contains information in Numeric value some value very high and some are very low if we see the age and estimated salary. This will cause some issues in our machinery model to solve that problem we set all values on the same scale there are two methods to solve that problem first one is Normalize and Second is Standard Scaler.

Standardisation	Normalisation
$x_{stand} = \frac{x - \text{mean}(x)}{\text{standard deviation } (x)}$	$x_{norm} = \frac{x - \min(x)}{\max(x) - \min(x)}$

Here we use Normalize import from Sklearn Library.

```
# feature Scaling
from sklearn.preprocessing import MinMaxScaler
sc = MinMaxScaler(feature_range = (0,1))
X = sc.fit_transform(X)
```

Part 2: Building & Train our Model

In this part, we model our Self Organizing Maps model.

2.1 Import the Model

In this step, we import our SOM models which are made by other developers. Link: https://test.pypi.org/project/MiniSom/1.0/

```
# import the SOM model
from minisom import MiniSom
```

2.2 Initialize our SOM model

In this step, we initialize our SOM model and we pass several parameters here. The first two are the dimension of our SOM map here **x= 10 & y= 10** mean we take 10 by 10 grid. The third parameter is **input length** we have 15 different attributes in our data set columns so input_lenght=15 here. The fourth parameter is **sigma** is the radius of a different neighborhood in the grid so we will keep 1.0 here which is the default value for SOMs. And last past parameters are **learning rate** which is hyperparameter the size of how much weight is updated during each iteration so higher is learning rate the faster is conversion and we keep the default value which is 0.5 here.

```
# init the model
som = MiniSom( x = 10, y = 10, input_len = 15, sigma = 1.0, learning_rate = 0.5)
```

2.3 Initialize the weights

In this step, we randomly initialize our weights from by using our SOM models and we pass only one parameter here which our data(X).

```
# init the weight
som.random_weights_init(X)
```

2.4 Train the Model

In this step we train our model here we pass two arguments here first is our data and the second is the number of iteration here we choose 100.

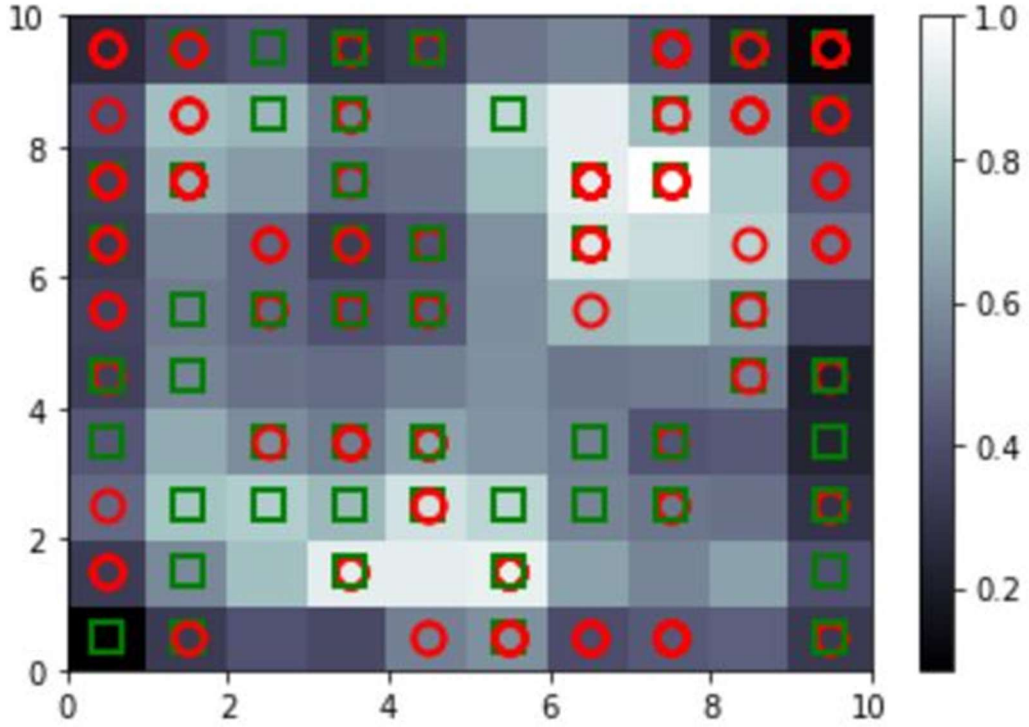
```
# traing the model
som.train_random(data = X, num_iteration = 100)
```

Part 3: Visualizing the result

In this step, we build a map of the Self Organizing Map. Firstly we import the library pylab which is used for the visualization of our result and we import different packages here. Bone is making a window then in the third line of code, we take a mean of all wining nodes. Then make of color bar which value is between 0 & 1. In Marker, we take a circle of red color which means the customer didn't get approval and square of green color which gets which customer gets approval. Then we make a for loop (i here correspond to index vector and x corresponds to customers) and inside for loop we take a wining node of each customer and this wining node is replaced by color marker on it and w[0] (x coordinate) and w[1] (y coordinate) are two coordinate) and then make a color of and take dependent variable which is 0 or 1 mean approval customer or didn't get approval and take a marker value of (o for red and s for green) and replace it.

```
# making a self organization map
from pylab import bone, pcolor, colorbar, plot, show
bone()
pcolor(som.distance_map().T)
colorbar()
markers = ['o' , 's']
colors = ['r', 'g']
for i, x in enumerate(X):
    w = som.winner(x)
    plot(w[0] + 0.5,
         w[1] + 0.5,
         markers[y[i]],
         markeredgecolor = colors[y[i]],
         markerfacecolor = 'None',
         markersize = 10,
         markeredgewidth = 2)
show()
```

Here is our Self Organizing map red circle mean customer didn't get approval and green square mean customer get approval. And if we look at our outlier then the white color area is high potential fraud which we detect here. And in the next part, we catch this cheater as you can see this both red and green.



Part 4: Catch the Potential Fraud

In this part, we catch the potential fraud of customer from the self-organizing map which we visualize in above

4.1 Mapping the wining node

In this step, we map all the wining nodes of customers from the Self Organizing Map.

4.2 Catch the cheater

In this step we catch the fraud to do that we take only those customer who potential cheat if we see in our SOM then clearly see that mapping [(7, 8), (3, 1) and (5, 1)] are potential cheat and use concatenate to concatenate of these three mapping values to put them in same one list.

```
#catch the cheater
frauds = np.concatenate((mappings[(7,8)], mappings[(3,1)], mappings[(5,1)] ) , axis=0)
```

4.3 Rescale the value

In this step, we convert our scale value into the original scale to do that we use the inverse function.

```
# rescale the value using inverse function
frauds = sc.inverse_transform(frauds)
```

4.4 Final results

Then simply call frauds and you get the whole list of those customers who potential cheat the bank.

```
frauds
```

EM algorithm:

Expectation and Maximization

Learning Objectives:

The learning objectives of this module are as follows:

- To understand the concept of Expectation Maximization (EM)
- To explain EM using a Coin Toss Example
- To understand the use of EM in K-means Algorithm

33.1 Introduction

The Expectation Maximization methodology was first presented in a general way by Dempster, Laird and Rubin in 1977. They define EM algorithm as an iterative estimation algorithm that can derive the maximum likelihood (ML) estimates in the presence of missing/hidden data ("incomplete data"). As an example we have theclassical case of the Gaussian mixture, where we have a set of unknown Gaussian distributions.The goal of the EM algorithm is to facilitate maximum likelihood parameter estimation by introducing so-called hidden random variables which are not observed and therefore define the unobserved data.

There are two main approaches to the applications of the EM algorithm. The first is when the data indeed has missing values, due to problems with or limitations of the observation process. The second approachis the optimization of the likelihood function is complex but however likelihood function can be simplified by assuming the existence of additional but missing (or hidden) parameters. The second type of application is more common in the pattern recognition. Now let us understand the concept of hidden and observed variables. Observed variables are directly measurable from the data, e.g. waveform values of a speech recording, Is it raining today? Did the smoke alarm go off?. On the other hand hidden variablesinfluence the data, but are not trivial to measure e.g includethe phonemes that produce a given speech recording, the probability P (rain today | rain yesterday) and is the smoke alarm malfunctioning?

Figure 33.1 Many to One Mapping

In Figure 33.1 X can be considered as the underlying space, x is the complete data (required for ML), *Y* is the observation space, y is the actual observation and x is observed only by means of y(x) and *X*(y) is a subset of X determined by y. The term y=(x) indicates that y is observed as some mapping of x.

The EM approach has found usage in filling in missing data in a sample, discovering the value of latent variables, estimating parameters of HMMs, estimating parameters of finite mixtures, unsupervised learning of clusters and finding parameters of Mixtures of Gaussians (MoG).

33.2 The EM algorithm

The steps of the EM algorithm are as follows:

1. We first consider a set of starting parameters given a set of incomplete (observed) data and we assume that observed data come from a specific model
2. We then use the model to "estimate" the missing data . In other words after formulating some parameters from observed data to build a model, we use this model to guess the missing value/data. This step is called the expectation step.
3. Now we use the "complete" data that we have estimated to update parameters whereusing the missing data and observed data, we find the most likely modified parameters to build the modified model. This is called the maximization step .
4. We repeat steps 2 & 3 until convergence that is there is no change in the parameters of the model and the estimated model fits the observed data.

The general idea of the algorithm is that we start by devising a noisy channel that is we use any model that predicts the corpus observations via some hidden structure and we Initially guess the parameters of the model. It is best to make an educated guess

but random assumptions can also work. Then we repeat until convergence the Expectation and Maximization steps. In the**Expectation step** we use current parameters (and observations) to reconstruct hidden structure while in the **Maximization step** weuse that hidden structure (and observations) to re-estimate parameters.

Figure 33.2 General Idea of the EM algorithm

In the example shown in Figure 33.2, we initially guess the probabilities of unknown parameters using which we guess the hidden structure such as tags, parses, weather (E step) and use it along with the observed structure such as words, eating ice cream to again re-estimate the probabilities (M step).

33.3 EM and Maximum Likelihood Estimates

As discussed above parameters describe the characteristics of a population. Their values are estimated from samples collected from that population.A maximum likelihood (ML) estimate is a parameter estimate that is most consistent with the sampled data since it attempts to maximize the likelihood function.

The basic setting of EM which is essentially based on maximum likelihood estimates is outlined below:

Let X be a set of data points considered as**observed** data and Θbethe parameter vector. Now EM is a method to find $\theta_{ML}$where $L(Q)$ is likelihood function

In essence we need to determine the probability of the observed data given the parameter vector. Now calculating P(X | θ) directly is hard. However calculating P(X,Y|θ) is much simpler, where Y is "hidden" data (or "missing" data) associated with the observed data.

Let us first define Z = (X, Y) where Z is considered as the complete data that is "augmented data", X is the observed data ("incomplete" data) and Y is the hidden data ("missing" data). EM is an iterative method to perform Maximum Likelihood estimation which starts with an initial estimate for θandrefines the current estimate iteratively to increase the likelihood of the observed data:

p(Z/θ)

33.4 EM – Coin Toss Example

The EM strategy can be explained with a coin toss example. This is the example we will be using in subsequent iterations to explain the complete flow of the EM algorithm. In this example we assume that we are tossing a number of coins sequentially to obtain a sequence of Head or Tails. The context of the coin toss example is given in Table 33.1. Here the problem is defined as X, the sequence of Heads and Tails that is observed, Y as the identifier of the coin that is tossed in the sequence, which is hidden and finally θwhich is the parameter vector which is associated with the probabilities ofthe observed and hidden data. Here if we assume three coins are tossed λ is the probability of coin 0 showing H (so 1 − λ is the probability of it showing T), p1 is the probability of coin 1 showing H, and p2 is the probability of coin 2 showing H.

Problem	Coin toss
X(observed)	Head-tail sequences

Y (hidden)	Coin id sequences
Θ	p1, p2, λ

Table 33.1 Parameters of EM

We can also modify the problem to figure out the probability of heads for two coins. Normally the ML estimate can be directly calculated from the results if we know the identity of which of the coins was tossed.

We have two coins indicated as coins**A** and **B**and let us assume that the probabilities for heads are **q*A*&q *B***respectively. We are given 5 measurements sets including 10 coin tosses in each set. Now we know which coin has been tossed in each measurement. The example sets of experiments are given in Table 33.2. In this table A coin has been indicated as red and B coin as blue. The first column in the table indicates the coin type for each of the 5 measurements since here we assume we know the identity of the coin. The second column indicates the sequence of 10 Heads and Tails observed for each measurement. Columns 3 and 4 indicate the number of Heads and Tails obtained in each coin toss for each measurement of each coin type. The final row shows the total number of Heads and Tails obtained for the measurements for each of the A and B coin types.

Table 33.2 Coin Toss Example of 5 measurements of 10 coin Tosses

Now we calculate the ML probabilities$\mathbf{q}_A$&$\mathbf{q}_B$, the probabilities for heads of coins A and B respectively.

Maximum Likelihoodof theprobabilities $\mathbf{q}_A$&$\mathbf{q}_B$arecalculated by dividing the total number of Heads obtained by the total number of Head and Tails observed for each type of coin. Thus we get

$\mathbf{q}_A = 24/24+6 = 24/30 = 0.8$

$\mathbf{q}_B = 9/9+11=9/20=0.45$

The above calculation is a basic probability calculation based on observations knowing whether coin A or B has been tossed.

We will now make the problem more interesting and assume that we do not even know which one of the coins is used for the sample set . Now we need to estimate the coin probabilities without knowing which one of the coins is being tossed.

33.5 EM Flow Explained with Coin Toss Example

Note that when we do not know which of the coins is tossed in each set we cannot calculate ML directly and hence we use EM strategy to find the probabilities of which one of the coins is likely to be tossed. Figure 33.3 shows the complete flow of the EM

algorithm. Remember we do not know which of the coins is tossed. Hence we start the process by assuming that for each of the coins A (red) and B (blue), the initial probabilities for heads are $\mathbf{q}_A$ & $\mathbf{q}_B$ respectivelywhich are assumed to have random values. Hence as seen from Figure 33.3 we have randomly fixed $\mathbf{q}_A$ to be 0.6 and $\mathbf{q}_B$ to be 0.5. Now we observe the number of Heads and Tails for each of the 5 measurements.

E Step:

The first stage is the **Expectation**stage for which we initially use the randomly assumed probabilities of $\mathbf{q}_A$ & $\mathbf{q}_B$ and the set of coin tosses observed for each measurement. Now we need to calculate the probabilities for Heads and Tails for both A and B coins for each measurement since we do not know which coin is tossed. We have shown the calculation for the first set .of measurements in Figure 33.3.

Step E-C1

In the first step of the **Expectation stage** we assume that the coin toss sequence follows a binomial distribution, where n is total number of coin tosses, k is number of Heads (Tails) observed and p is the probability of observing heads for each coin.

Figure 33.3 Flow of EM – Coin Toss Example

Usingthe distribution we can calculate the probability of observing Heads and Tails for coins A and B for the first set as shown in Figure 33.4. Here

n – the total number of coins tossed in the first measurement = **10,** k -the total number of heads observed =**5**

Now we need to calculate the probability of observing Heads(Tails) for both coins A and B since we do not know which of the coins was tossed and hence p – (q_A)- the probability of heads for A (red) coin is initially assumed to be = **0.6** & (q_b)- the probability of heads for B (blue) coin is initially assumed to be = **0.5**

Figure 33.4 Use of Binomial Distribution

Step E-C2

In the second step of the Expectation stage we calculate the probabilities of using coin A or B as follows:

0.201/ (0.201+0.246)=0.45

0.246/ (0.201+0.246)=0.55

We calculate in a similar manner for all the experiments and the values obtained are shown in Figure 33.3.

Step E-C3

In the third step of the Expectation stage using the probability values obtained in step E-C2, we calculate the possible number of Heads and Tails that is likely to be observed in each experiment if the coin tossed was A and if the coin tossed was B

First Experiment – values corresponding to first row:

(i) 5(number of heads in First experiment)

*0.45(probability of tossing coin A)=2.2H,

(ii) 5(number of Tails in First experiment)

*0.45(probability of tossing coin A)=2.2H,

(iii) 5(number of heads in First experiment) *0.55(probability of tossing coin B)=2.8H, (

iv)5(number of Tails in First experiment) *0.55(probability of tossing coin B)=2.8H.

We will also explain the calculations for second experiment .

Second Experiment – values corresponding to second row:

(i) 9(number of heads in Second experiment)

*0.80(probability of tossing coin A)=7.2H,

(ii) 1(number of Tails in Second experiment)

*0.8(probability of tossing coin A)=0.8H,

(iii) 9(number of heads in Second experiment) *0.20(probability of tossing coin B)=1.8H, (iv)1(number of Tails in Second experiment)

*0.20(probability of tossing coin B)=0.2H.

Similarly we can do the calculations for all 5 experiments. Using these calculated values for number of Head and Tails for each experiment, we can calculate the total number of Heads and Tails for both Coins A and B.

M Step

Now we have the Maximization stage where we calculate the new values (after 1st iteration) of $\mathbf{q}_{\boldsymbol{A\ B(1)}}$ that is the maximum likelihood estimates of the probability

of heads when coin A and probability of heads when coin B are tossed respectively using the values total number of Heads and Tails for both Coins A and B. This calculation is shown below:

q*A(1)*= 21.3(total number of Heads when coin A is tossed)/(21.3+8.6)(total number of Heads and Tails when coin A is tossed)= **0.71**

q *B(1)*= 11.7(total number of Heads when coin B is tossed)/(11.7+8.4)(total number of Heads and Tails when coin B is tossed)=**0.58**

Continuing and Completing the EM algorithm

Now we have completed one iteration of the EM algorithm. We now continue the second iteration of the algorithm using the above new values of **q*A*&q *B***. We continue the iterations until the values of **q*A*&q *B*** do not change from one iteration to the next. This happens in the 10th iteration for our example (shown in Figure 33.3) when the values **q*A*&q *B*** converge to values **0.80** and **0.52** respectively.

33.6 Kmeans and EM algorithm

We can explain K means as an EM algorithm. First we initialize the k means (mk) of the Kmeans algorithm. In the E Step weassign each point to a Cluster and during the M Step given the Clusters we refine mean mkof each cluster k. This process is repeated until the change in means is small.

33.6.1 Generating Data from Mixture of Gaussians

Now we can replace the ‘hard’ clustering of K-means described above with ‘soft’ probabilistic assignments . Here we assume that each instance x is generated

Figure 33.5 The Gaussian Distributions used to Generate Data

Figure 33.6 The Use of a Mixture of Gaussians for Generating Data

by choosing one of the k Gaussians at random and generating an instance according to that Gaussian(Figure 33.5). This requires more parameters to be determined that would fit the data. Figure 33.6 shows the probability of generating data p(X). This probability is based on the Gaussian distributions used, the mean and the variance of the Gaussian distributions and the mixing coefficients used. The sum of the mixing coefficients of all the Gaussian distributions use must be 1.

33.6.2 K-means and Mixture of Gaussians

Now we know that in a general K-means which is essentially a classifier and we need to find the parameterto fit data – that is we need to find the mean – μk as already discussed above. However when we use mixture of Gaussians which is a probability model where we are defining a "soft" classifier. Now the parameters that are to be determined to fit to data are the means μk*and* *c*ovariance Σkwhich define the Gaussians distributions and the mixing coefficient πk. Now given the data set, find the mixing coefficients, means and covariance. If we knew which component generated each data point, the maximum likelihood solution would involve fitting each component to the corresponding cluster . However our problem is that the data set is unlabelled or are hidden (Figure 33.7).

Figure 33.7 K Means Scenario

Figure 33.8 K means with Initial Guess of Cluster Points 33.6.3 EM for Estimating k Means

Figure 33.8 shows the scenario of K means the instances from X are generated by a mixture of k Gaussians with unknown means <m1,…,mk> of the k Gaussians.Remember we do not know which instance xi was generated by which Gaussian. Now we need to determine the maximum likelihood estimates of <m1,…,mk>. Now let us define the full description of each instance as yi=<xi,zi1,zi2> where zij is 1 if xi generated by j-th Gaussian. In this case xiis observable but however zijis unobservable.

33.6.4 EM for Gaussian Mixtures

Here we initialize the Gaussian parameters mean mk, co-variance å k and mixing coefficient pk.

E Step:

In the E Step we assign each point xn an assignment score g(znk) for each cluster or Gaussian kwhich essentially indicates how much this Gaussian k is responsible for point Xn

Figure 33.9 Calculation of Assignment Score

Here g(znk) is calculated as the mean and covariance of the Gaussian distribution corresponding to the kth mean multiplied by mixture coefficient of kth distribution divided by the summation of mean and covariance of all the Gaussian distributions multiplied by their corresponding mixture coefficient.

M step

During the M Step, given scores, adjust mk, å k, pk for each cluster kor Gaussian K. We update parameters using new g(znk) that is find the parameters that fit the new assignment score g(znk) the best. The new values of each of the parameters mknew , å knew and pknew are determined as shown in Figure 33.10.

Figure 33.10 Calculating new Values of Parameters during M Step

Now we evaluate log likelihood as shown in Figure 33.11 . If likelihood or parameters converge we stop else we iterate with the E and M steps.

Figure 33.11 Evaluation of Log Likelihood

33.7 Strengths of EM

The major strength of the EM algorithm is its numerical stability where in every iteration of the EM algorithm, the likelihood of the observed data increases that is we are heading towards a solution. In addition, the EM handles parameter constraints gracefully.

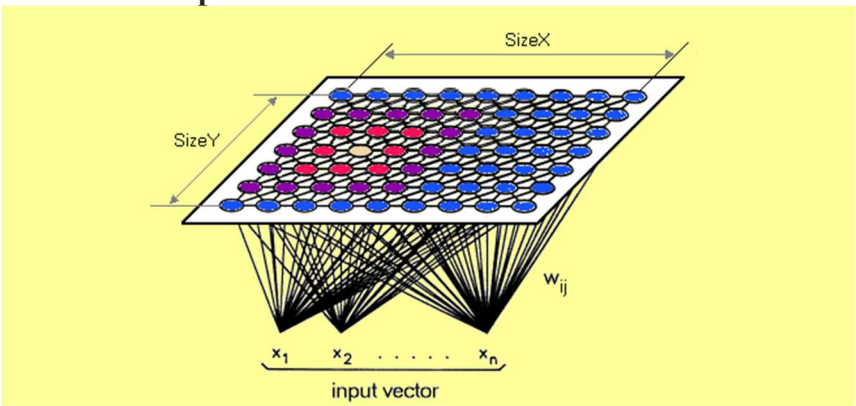
33.8 Problems with EM

In the case of EM algorithms can converge very slowly on some problems and this convergence is intimately related to the amount of missing information.It guarantees to improve the probability of the training corpus, which is different from reducing the errors directly. The EM algorithm cannot guarantee to reach global maximum and sometimes could get struck at the local maxima, saddle points, etc. Essentially the guess we make of the initial parameter values is very important and can decide on the time to converge.

A **self-organizing map** (**SOM**) is a type of artificial neural network (ANN) that is trained using unsupervised learning to produce a low-dimensional (typically two-dimensional), discretized representation of the input space of the training samples, called a **map**, and is therefore a method to do dimensionality reduction. Self-organizing maps differ from other artificial neural networks as they apply competitive learning as opposed to error-correction learning (such as

backpropagation with gradient descent), and in the sense that they use a neighborhood function to preserve the topological properties of the input space.

SOM was introduced by Finnish professor Teuvo Kohonen in the 1980s is sometimes called a **Kohonen map.**



Dimensionality reduction in SOM

What really happens in SOM ?

Each data point in the data set recognizes themselves by competeting for representation. SOM mapping steps starts from initializing the weight vectors. From there a sample vector is selected randomly and the map of weight vectors is searched to find which weight best represents that sample. Each weight vector has neighboring weights that are close to it. The weight that is chosen is rewarded by being able to become more like that randomly selected sample vector. The neighbors of that weight are also rewarded by being able to become more like the chosen sample vector. This allows the map to grow and form different shapes. Most generally, they form square/rectangular/hexagonal/L shapes in 2D feature space.

Inference:

If the average distance is high, then the surrounding weights are very different and a light color is assigned to the location of the weight. If the average distance is low, a darker color is assigned. The resulting maps show that the concentration of different clusters of species are more predominant in three zones. First figure tells us only about where the density of species is greater (darker regions) or less (lighter regions). The second visualisation tells us how they are specifically clustered.

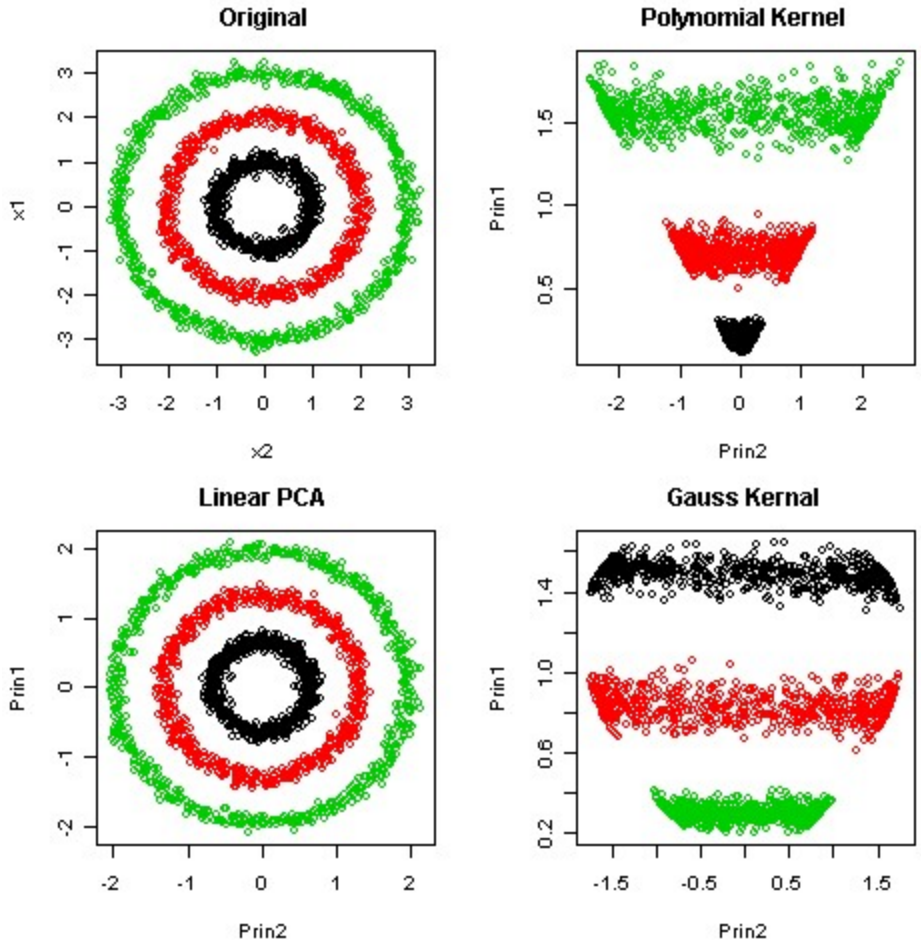
Cons of Kohonen Maps:

1. It does not build a generative model for the data, i.e, the model does not understand how data is created.
2. It does not behave so gently when using categorical data, even worse for mixed types data.
3. The time for preparing model is slow, hard to train against slowly evolving data.

PCA----------

Kernel PCA: An In-Depth Explanation

Kernel Principal Component Analysis (Kernel PCA) is an extension of Principal Component Analysis (PCA) that allows for the analysis of data that is not linearly separable. By using a kernel function to map data into a higher-dimensional space, Kernel PCA can uncover complex structures within the data that traditional PCA might miss.



PCA and Kernal PCA

How Kernel PCA Works

1. **Data Mapping with Kernel Trick**:

- Kernel PCA maps the original data into a higher-dimensional feature space using a kernel function.
- The kernel function k(xi,xj) computes the inner products between the images of the data points in the feature space, without explicitly computing the coordinates of the points in that space (the “kernel trick”).

2. Compute the Kernel Matrix:

- Construct a symmetric kernel matrix K where Kij=k(xi,xj).

3. Center the Kernel Matrix:

- Center the kernel matrix to ensure that the data has zero mean in the feature space. This is done using the formula:

$$K' = K - 1_N K - K 1_N + 1_N K 1_N$$

where 1N is a matrix of ones divided by N (the number of data points).

4. Compute Eigenvalues and Eigenvectors:

- Perform eigenvalue decomposition on the centered kernel matrix K′. Let $\lambda_1, \lambda_2, \ldots, \lambda_N$ be the eigenvalues and $\alpha_1, \alpha_2, \ldots, \alpha_N$ be the corresponding eigenvectors.

5. Project Data onto Principal Components:

- The principal components are the projections of the data onto the eigenvectors in the feature space. The transformed data in the new feature space is given by:

$$\mathbf{z_i} = \sum_{j=1}^{N} \alpha_j k(x_i, x_j)$$

for each data point x_i.

Differences Between PCA and Kernel PCA

Linear vs. Non-Linear:

- PCA is a linear method and is only capable of capturing linear relationships in the data.
- Kernel PCA, on the other hand, can capture non-linear relationships by implicitly mapping the data into a higher-dimensional space using a kernel function.

Feature Space:

- In PCA, the data is transformed within the original feature space.

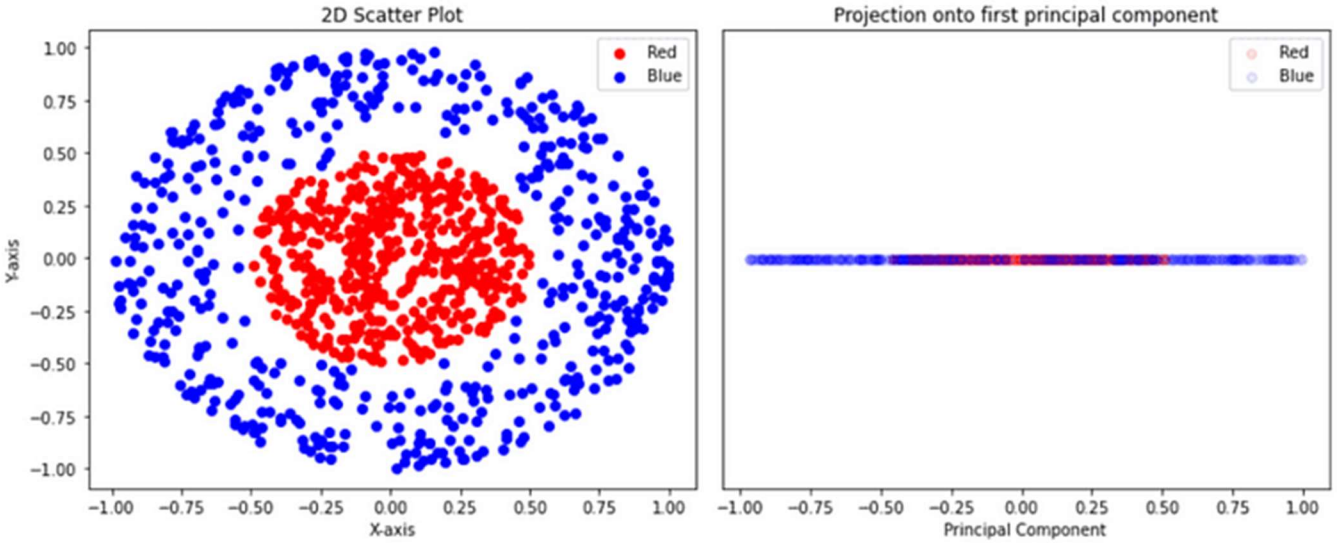
- In Kernel PCA, the data is transformed in a high-dimensional feature space determined by the kernel function.

Kernel Function:

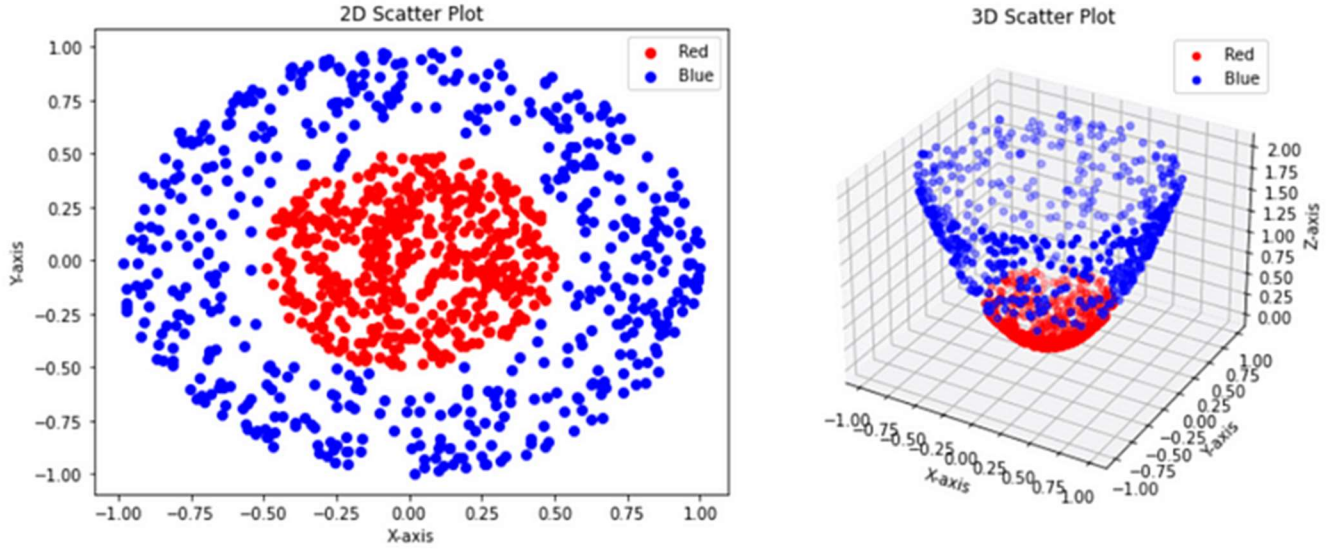
- PCA does not use a kernel function.
- Kernel PCA uses kernel functions (e.g., polynomial, Gaussian RBF) to compute the inner products in the high-dimensional feature space.

Working of Kernel PCA: Step-by-Step

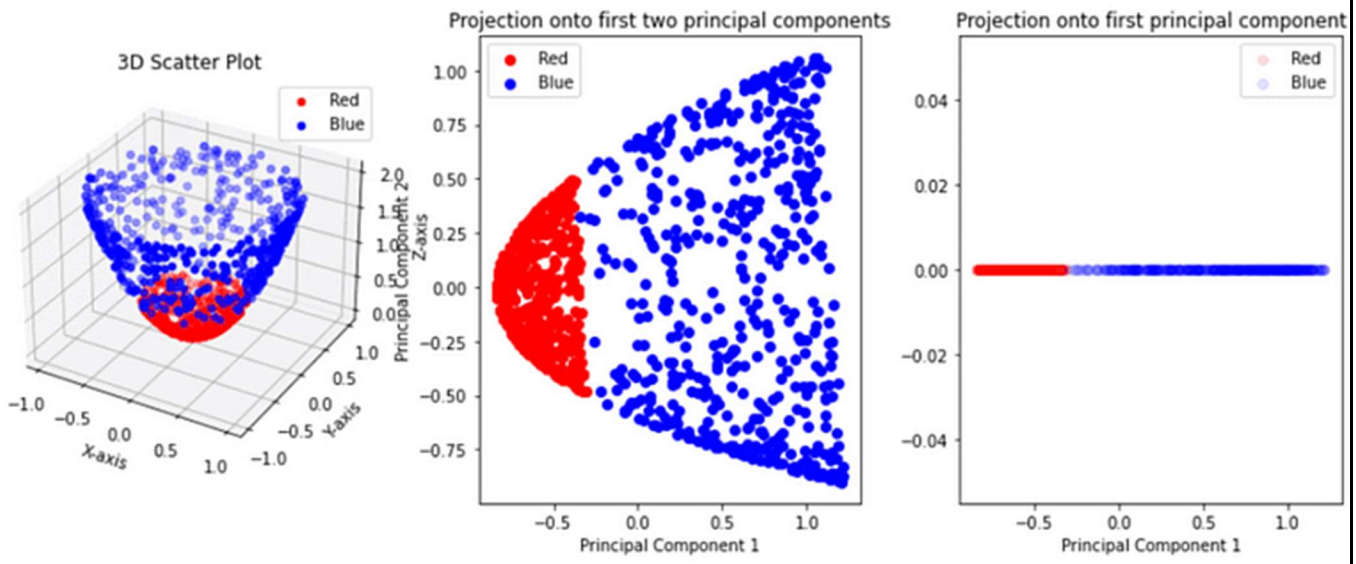
1. **Select a Kernel**: Choose a kernel function (e.g., polynomial, Gaussian RBF) based on the nature of the data and the problem at hand.
2. **Construct the Kernel Matrix**: Compute the kernel matrix KKK using the chosen kernel function for all pairs of data points.
3. **Center the Kernel Matrix**: Center the kernel matrix to have zero mean in the feature space.
4. **Eigenvalue Decomposition**: Perform eigenvalue decomposition on the centered kernel matrix to find the eigenvalues and eigenvectors.
5. **Project Data**: Project the original data onto the principal components (eigenvectors) in the high-dimensional feature space.



Data in 2D shows data is not linearly separable



Data in 3D



Now we can separate the data linearly.

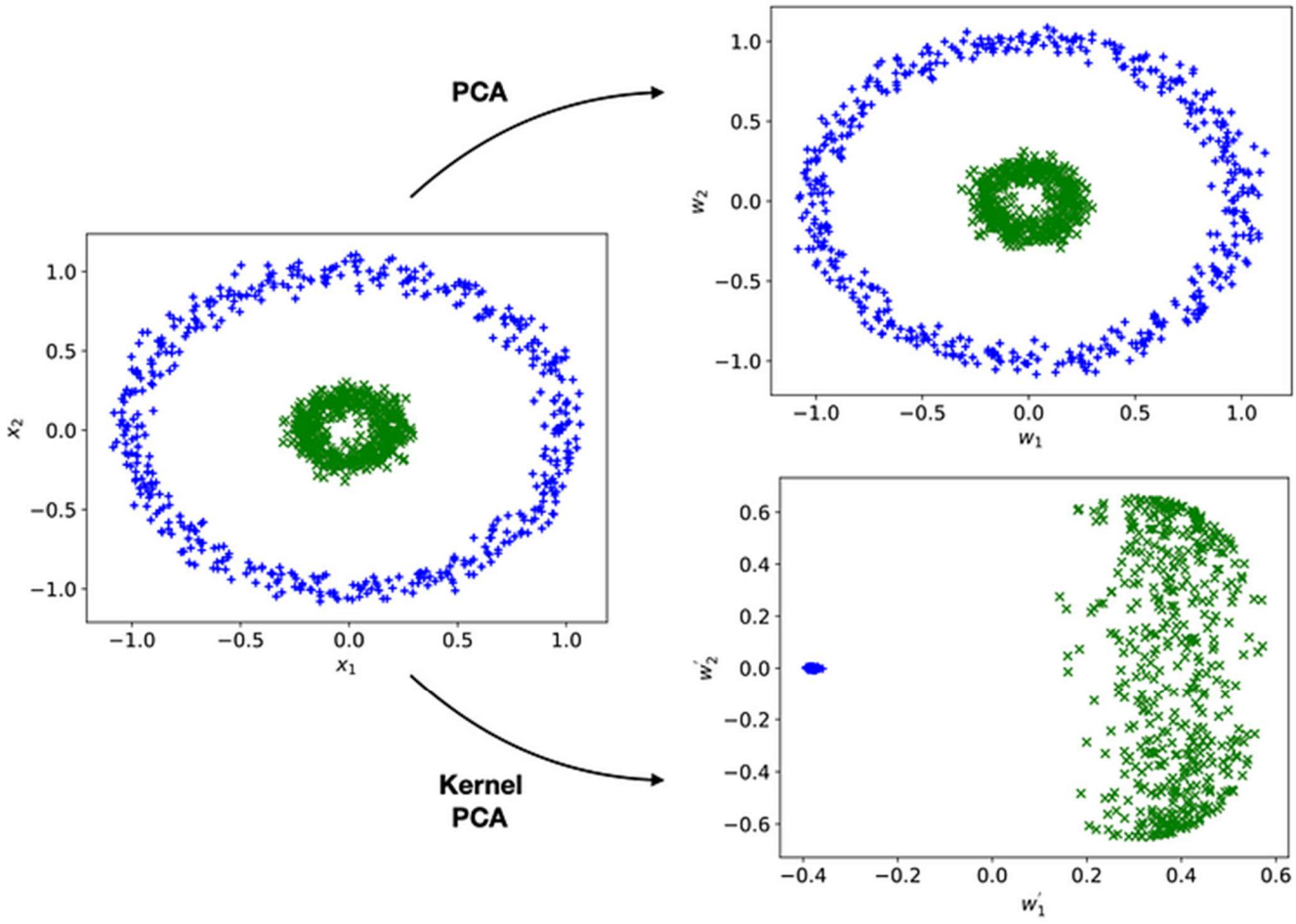
Pros and Cons of Kernel PCA

Pros:

- **Non-Linear Data**: Capable of handling non-linear data structures and capturing complex patterns.
- **Flexibility**: Various kernel functions can be used to adapt to different types of data and problems.
- **Higher Dimensional Insights**: Allows for the analysis of data in higher-dimensional spaces without explicitly computing the coordinates.

Cons:

- **Computationally Intensive**: Kernel PCA can be more computationally intensive than linear PCA, especially for large datasets.
- **Choice of Kernel**: The performance of Kernel PCA heavily depends on the choice of the kernel function and its parameters.
- **Interpretability**: The results of Kernel PCA can be harder to interpret compared to linear PCA, especially when using complex kernel functions.



PCA vs Kernal PCA

Common Kernel Functions in Kernel PCA

1. **Linear Kernel**
2. **Polynomial Kernel**
3. **Radial Basis Function (RBF) Kernel**
4. **Sigmoid Kernel**

When to Use Different Kernel Functions

- **Linear Kernel**: Use when the data is linearly separable or when you suspect that a linear relationship might be sufficient.
- **Polynomial Kernel**: Use when the data has polynomial relationships of a certain degree.
- **RBF Kernel**: Use when the data is not linearly separable, and you suspect the data can be separated with a non-linear boundary.
- **Sigmoid Kernel**: Use in cases where you want to mimic the behaviour of neural networks.

Kernel Type	Formula	Use Case	Pros	Cons
Linear Kernel	$k(x, y) = x \cdot y$	When data is linearly separable or close to linear separability.	Simple and fast; works well for linearly separable data.	Cannot capture non-linear relationships.
Polynomial Kernel	$k(x, y) = (x \cdot y + c)^d$	When data has polynomial relationships of a certain degree.	Can capture polynomial relationships; flexible with degree d.	Computationally expensive for high-degree polynomials; can overfit.
RBF Kernel (Gaussian Kernel)	$k(x, y) = \exp\left(-\frac{\|x-y\|^2}{2\sigma^2}\right)$	When data is not linearly separable; non-linear boundaries are needed.	Can capture very complex relationships; widely used and effective.	Requires careful tuning of σ; computationally intensive for large datasets.
Sigmoid Kernel	$k(x, y) = \tanh(\alpha x \cdot y + c)$	In scenarios mimicking neural network behavior.	Can capture certain non-linear relationships; inspired by neural networks.	Not as popular or effective as RBF in many scenarios; requires careful tuning of α and c.

Different types of Kernal PCA

t-SNE

What is t-SNE?

(t-SNE) t-Distributed Stochastic Neighbor Embedding is a non-linear dimensionality reduction algorithm for exploring high-dimensional data. It maps multi-dimensional data to two or more dimensions suitable for human observation. With the help of the t-SNE algorithms, you may have to plot fewer exploratory data analysis plots next time you work with high-dimensional data.



What is Dimensionality Reduction?

To understand how t-SNE works and t-SNE visualization, let's first understand what dimensionality reduction is.

In simple terms, dimensionality reduction represents multidimensional data (data with multiple features correlating with each other) in two or three dimensions.

Some of you might question why we need dimensional reduction when we can plot the data using scatter plots, histograms & boxplots and make sense of the pattern in data using descriptive statistics.

Well, even if you can understand the data patterns and present them on simple charts, it is still difficult for anyone without a statistics background to understand them. Also, if you have hundreds of features, you must study thousands of charts before making sense of this data. (Read more about dimensionality reduction here)

With the help of a dimensionality reduction algorithm, you can present the data explicitly.

How does t-SNE Work, and How Can it be Implemented?

t-SNE, or t-Distributed Stochastic Neighbor Embedding, is a method for converting Visualize high-dimensional data into a more manageable form, typically 2D or 3D, facilitating better comprehension. Here's how it operates:

1. **Measure Similarity**: Initially, it assesses the similarity between each pair of data points within the high-dimensional space.
2. **Map to Lower Dimensions**: Subsequently, it endeavors to represent these similarities in a simplified space while preserving the relationships between the points as faithfully as possible.
3. **Adjust Positions**: It iteratively adjusts the positions of points in the simplified space to align the similarities as closely as feasible.

4. **Visualize**: Finally, it plots these points in the simplified space, often on a graph, facilitating easier pattern recognition and analysis.

To utilize t SNE, one must prepare the data point, execute the t-SNE algorithm (accessible in libraries like scikit-learn or TensorFlow), and visualize the outcomes. It is a valuable tool for comprehending intricate datasets.

How does t-SNE fit in the dimensionality reduction algorithm space?

Now that you understand what dimensionality reduction is, let's look at how we can use the t-SNE algorithm to reduce low dimensional points.

Following are a few dimensionality reduction algorithms that you can check out:

1. PCA (linear)
2. t SNE (non-parametric/ nonlinear)
3. Sammon mapping (nonlinear)
4. Isomap (nonlinear)
5. LLE (nonlinear)
6. CCA (nonlinear)
7. SNE (nonlinear)
8. MVU (nonlinear)
9. Laplacian Eigenmaps (nonlinear)

The good news is that you need to study only two of the abovementioned algorithms to visualize data in lower dimensions – PCA and t SNE effectively.

Limitations of PCA

PCA is a linear algorithm. It cannot interpret complex datasets of polynomial relationships between features. On the other hand, t-SNE machine learning is based on probability distributions with random walks on neighborhood graphs to find the structure of the data.

A major problem with linear dimensionality reduction algorithms is that they concentrate on placing dissimilar data points far apart in a lower-dimension representation. However, to represent high-dimension data on a dimension, non-linear manifold, similar data points must be represented close together, which is not what linear dimensionality reduction algorithms do.

Now, you have a brief understanding of what PCA endeavors to do.

Local approaches seek to map nearby points on the manifold to nearby points in the low-dimensional representation. Global approaches, on the other hand, attempt to preserve geometry at all scales, i.e, mapping nearby points to nearby points and far away points to far away points

It is important to know that most nonlinear techniques other than t-SNE cannot simultaneously retain the local and global structure of the data.

Algorithmic details of t-SNE (optional read)

This section is for people interested in understanding the algorithm in depth. You can safely skip this section if you do not want to review the math thoroughly.

Let's understand why you should know about t-SNE and its algorithmic details. t-SNE is an improvement on the Stochastic Neighbor Embedding (SNE) algorithm.

Algorithm

Step 1

Stochastic Neighbor Embedding (SNE) starts by converting the high-dimensional Euclidean distances between data points into conditional probabilities representing similarities. The similarity of datapoint x_i to datapoint x_j is the conditional probability, $p_{j|i}$, x_i would pick x_j as its neighbor if neighbors were picked in proportion to their probability density under a Gaussian centered at x_i.

For nearby datapoints, $p_{j|i}$ is relatively high, whereas for widely separated datapoints, $p_{j|i}$ will be almost infinitesimal (for reasonable values of the variance of the Gaussian, σ_i). Mathematically, the conditional probability $p_{j|i}$ is given by

$$p_{j|i} = \frac{\exp\left(-\|x_i - x_j\|^2 / 2\sigma_i^2\right)}{\sum_{k \neq i} \exp\left(-\|x_i - x_k\|^2 / 2\sigma_i^2\right)},$$

where σ_i is the variance of the Gaussian that is centered on datapoint x_i

If you are not interested in the math, think about it in this way, the algorithm starts by converting the shortest distance (a straight line) between the points into probability of similarity of points. Where, the similarity between points is: the

conditional probability that x_i would pick x_j as its neighbor if neighbors were picked in proportion to their probability density under a Gaussian distribution centered at x_i.

Step 2

For the low-dimensional counterparts y_i and y_j of the high-dimensional datapoints x_i and x_j it is possible to compute a similar conditional probability, which we denote by $q_{j|i}$.

$$q_{j|i} = \frac{\exp\left(-\|y_i - y_j\|^2\right)}{\sum_{k \neq i} \exp\left(-\|y_i - y_k\|^2\right)}.$$

Note that, pi|i and pj|j are set to zero as we only want to model pair wise similarity.

In simple terms step 1 and step2 calculate the conditional probability of similarity between a pair of points in

1. High dimensional space
2. In low dimensional space

For the sake of simplicity, try to understand this in detail.

Let us map 3D space to 2D space. What step1 and step2 are calculating the probability of similarity of points in 3D space and calculating the probability of similarity of points in the corresponding 2D space.

Logically, the conditional probabilities $p_{j|i}$ and $q_{j|i}$ must be equal for a perfect representation of the similarity of the datapoints in the different dimensional spaces, i.e the difference between $p_{j|i}$ and $q_{j|i}$ must be zero for the perfect replication of the plot in high and low dimensions.

By this logic SNE attempts to minimize this difference of conditional probability.

Step 3

Now here is the difference between the SNE and t-SNE algorithms.

To measure the sum of the differences of conditional probability, SNE minimizes the sum of Kullback-Leibler divergence's overall data points using a gradient descent method. We must know that KL divergences are asymmetric.

In other words, the SNE cost function focuses on retaining the local structure of the data in the map (for reasonable values of the variance of the Gaussian in the high-dimensional space, σ_i).

Additionally, it is very difficult (computationally inefficient) to optimize this cost function.

So, t SNE also tries to minimize the sum of the difference in conditional probabilities. But it uses the symmetric SNE cost function version with simple gradients. Also, t-SNE machine learning employs a heavy-tailed distribution in the low-dimensional space to alleviate both the crowding problem (the area of the two-dimensional map that is available to accommodate moderately distant data points

will not be nearly large enough compared with the area available to accommodate nearby data points) and the optimization problems of SNE.

Step 4

If we see the equation to calculate the conditional probability, we have left out the variance from the discussion. The remaining parameter to be selected is the variance σ_i of the student's t-distribution centered over each high-dimensional datapoint x_i. It is not likely that there is a single value of σ_i that is optimal for all data points in the data set because the density of the data is expected to vary. In dense regions, a smaller value of σ_i is usually more appropriate than in sparser areas. Any particular value induces a probability distribution, P_i , over all of the other data points. This distribution has an

This distribution has an entropy which increases as σ_i increases. t-SNE machine learning performs a binary search for the value of σ_i that produces a P_i with a fixed perplexity that is specified by σ_i the user. The perplexity is defined as

$$Perp(P_i) = 2^{H(P_i)},$$

where H(P_i) is the Shannon entropy of P_i measured in bits

$$H(P_i) = -\sum_j p_{j|i} \log_2 p_{j|i}.$$

The perplexity can be interpreted as a smooth measure of the effective number of neighbors. The performance of SNE is fairly robust to changes in the perplexity, and typical values are between 5 and 50.

The minimization of the cost function is performed using gradient descent algorithm. And physically, the gradient may be interpreted as the resultant force created by a set of springs between the map point y_i and all other map points y_j . All springs exert a force along the direction (y_i – y_j). The spring between y_i and y_j repels or attracts the map points depending on whether the distance between the two in the map is too small or too large to represent the similarities between the two high-dimensional datapoints. The force exerted by the spring between y_i and y_j is proportional to its length, and also proportional to its stiffness, which is the mismatch (pj|i – qj|i + p i| j − q i| j) between the pairwise similarities of the data points and the map points[1].-

Time and Space Complexity

Now that we have understood the algorithm, it is time to analyze its performance. As you might have observed, the algorithm computes pairwise conditional probabilities and tries to minimize the sum of the difference of the probabilities in higher and lower dimensions. This involves a lot of calculations and computations. So, the algorithm is quite heavy on the system resources.

t-SNE has a quadratic time and space complexity regarding the number of data points. This makes it particularly slow and resource-draining when applied to data sets comprising more than 10,000 observations.

What Does t-SNE Algorithm Do?

After we have looked into the mathematical description of how the algorithms works, to sum up what we have learned above. Here is a brief explanation of how t-SNE works.

It's quite simple: t-sne, a non-linear dimensionality reduction algorithm, finds patterns in the data by identifying observed clusters based on the similarity of data points with multiple features. However, it is not a clustering algorithm but a dimensionality reduction algorithm. It maps the multi-dimensional data to a lower-dimensional space, and the input features are no longer identifiable. Thus, you cannot make any inference based only on the output of t-SNE machine learning. So, essentially, it is mainly a data exploration and visualization technique.

However, t-SNE can be used in classification and clustering by using its output as the input feature for other classification algorithms.

Use Cases

You may ask, what are the use cases of such an algorithm? t-SNE can be used on almost all high-dimensional data sets. However, it is extensively applied in image processing, NLP, genomic data, and speech processing. It has been utilized to improve the analysis of brain and heart scans. Below are a few examples:

Facial Expression Recognition

Much progress has been made on FER, and many algorithms like PCA have been studied for FER. However, FER remains a challenge due to dimension reduction

and classification difficulties. t-Stochastic Neighbor Embedding (t SNE) is used to reduce the high-dimensional data into a relatively low-dimensional subspace and then use other algorithms like AdaBoostM2, Random Forests, Logistic Regression, Neural Networks, and others as multi-classifiers for expression classification.

In a recent endeavor focused on facial recognition utilizing the Japanese Female Facial Expression (JAFFE) database, data visualization techniques, specifically t-SNE dimensionality reduction, were employed alongside the AdaBoostM2 algorithm. The experimental findings indicated that the newly proposed algorithm, integrated with t SNE and AdaBoostM2, outperformed traditional approaches like PCA, LDA, LLE, and SNE. This highlights the efficacy of advanced data visualization methods, particularly t-SNE, in enhancing facial expression recognition (FER) systems. Additionally, this project underscores the potential benefits of incorporating such techniques into partner programs to advance facial recognition technology.

The flowchart for implementing such a combination of the data could be as follows:

Preprocessing → normalization → t-SNE→ classification algorithm

	PCA	LDA	LLE	SNE	t-SNE
SVM	73.5%	74.3%	84.7%	89.6%	90.3%
AdaboostM2	75.4%	75.9%	87.7%	90.6%	94.5%

Identifying Tumor subpopulations (Medical Imaging)

Mass spectrometry imaging (MSI) is a technology that simultaneously provides the spatial distribution for hundreds of biomolecules directly from tissue. Spatially mapped t-distributed stochastic neighbor embedding (t-SNE) is a nonlinear visualization of the data that can better resolve the biomolecular intratumor heterogeneity.

In an unbiased manner, t-SNE machine learning can uncover tumor subpopulations that are statistically linked to patient survival in gastric cancer and metastasis status in primary tumors of breast cancer. Survival analysis performed on each t-SNE cluster will provide significantly useful results.[3]

Text Comparison Using Word2vec

Word vector representations capture many linguistic properties such as gender, tense, plurality, and even semantic concepts like “capital city of”. Using dimensionality reduction, a 2D map can be computed where semantically similar words are close to each other. This combination of techniques can provide a bird’s-eye view of different text sources, including text summaries and their source material. This enables users to explore a text source like a geographical map.[4]

Comparison of t-SNE to Other Dimensionality Reduction Algorithms

While comparing the performance of t-SNE with other algorithms, we will compare t-SNE with other algorithms based on the achieved accuracy rather than the time and resource requirements in relation to accuracy.

t-SNE outputs provide better results than PCA and other linear dimensionality reduction models. This is because a linear method such as classical scaling is not good at modeling curved manifolds. It focuses on preserving the distances between widely separated data points rather than the distances between nearby data points.

The Gaussian kernel employed in the high-dimensional space by t-SNE defines a soft border between the local and global structure of the data. For pairs of data points that are close together relative to the standard deviation of the Gaussian, the importance of modeling their separations is almost independent of the magnitudes of those separations. Moreover, t-SNE machine learning determines the local neighborhood size for each data point separately based on the local density of the data (by forcing each conditional probability distribution to have the same perplexity)[1]. This is because the algorithm defines a soft border between the local and global structure of the data. And unlike other non-linear dimensionality reduction algorithms, it performs better than any.

Implementation Using Examples

Let’s implement the t-SNE algorithm on the MNIST handwritten digit database. This is one of the most explored datasets for image processing.

> MNIST data can be downloaded from the MNIST website and can be converted into a csv file with a small amount of code.
>
> For this example, please download the following preprocessed MNIST data. link

```
## calling the installed package
train<- read.csv(file.choose()) ## Choose the train.csv file downloaded from the link above
library(Rtsne)
## Curating the database for analysis with both t-SNE and PCA
Labels<-train$label
train$label<-as.factor(train$label)
## for plotting
colors = rainbow(length(unique(train$label)))
names(colors) = unique(train$label)

## Executing the algorithm on curated data
tsne <- Rtsne(train[,-1], dims = 2, perplexity=30, verbose=TRUE, max_iter = 500)
exeTimeTsne<- system.time(Rtsne(train[,-1], dims = 2, perplexity=30, verbose=TRUE, max_iter = 500))

## Plotting
plot(tsne$Y, t='n', main="tsne")
text(tsne$Y, labels=train$label, col=colors[train$label])
```
Copy Code

Implementation In Python

An important thing to note is that the “pip install tsne” produces an error. Installing “tsne” package is not recommended. t-SNE algorithm can be accessed from sklearn package.

- Hyperparameter tuning

n_components : int, optional (default: 2)	Dimension of the embedded space.
perplexity : float, optional (default: 30)	The perplexity is related to the number of nearest neighbors that is used in other manifold learning algorithms. Larger datasets usually require a larger perplexity. Consider selcting a value between 5 and 50. The choice is notn extremely critical since t-SNE is quite insensitive to this parameter.
early_exaggeration : float, optional (default: 4.0)	Controls how tight natural clusters in the original space are in the embedded space and how much space will be between them. For larger values, the space between natural clusters will be larger in the embedded space. Again, the choice of this parameter is not very critical. If the cost function increases during initial optimization, the early exaggeration factor or the learning rate might be too high.
learning_rate : float, optional (default: 1000)	The learning rate can be a critical parameter. It should be between 100 and 1000. If the cost function increases during initial optimization, the early exaggeration factor or the learning rate might be too high. If the cost function gets stuck in a bad local minimum increasing the learning rate helps sometimes.
n_iter : int, optional (default: 1000)	Maximum number of iterations for the optimization. Should be at least 200.
metric : string or callable, (default: "euclidean")	The metric to use when calculating distance between instances in a feature array. If metric is a string, it must be one of the options allowed by scipy.spatial.distance.pdist for its metric parameter, or a metric listed in pairwise.PAIRWISE_DISTANCE_FUNCTIONS If metric is "precomputed", X is assumed to be a distance matrix. Alternatively, if

- Code

The following code is taken from the sklearn examples on the sklearn website.

Python Code:

```
from time import time
import numpy as np
import matplotlib.pyplot as plt
from matplotlib import offsetbox
from sklearn import (manifold, datasets, decomposition, ensemble,
         discriminant_analysis, random_projection)
## Loading and curating the data
digits = datasets.load_digits(n_class=10)
X = digits.data
y = digits.target
n_samples, n_features = X.shape
n_neighbors = 30
## Function to Scale and visualize the embedding vectors
```

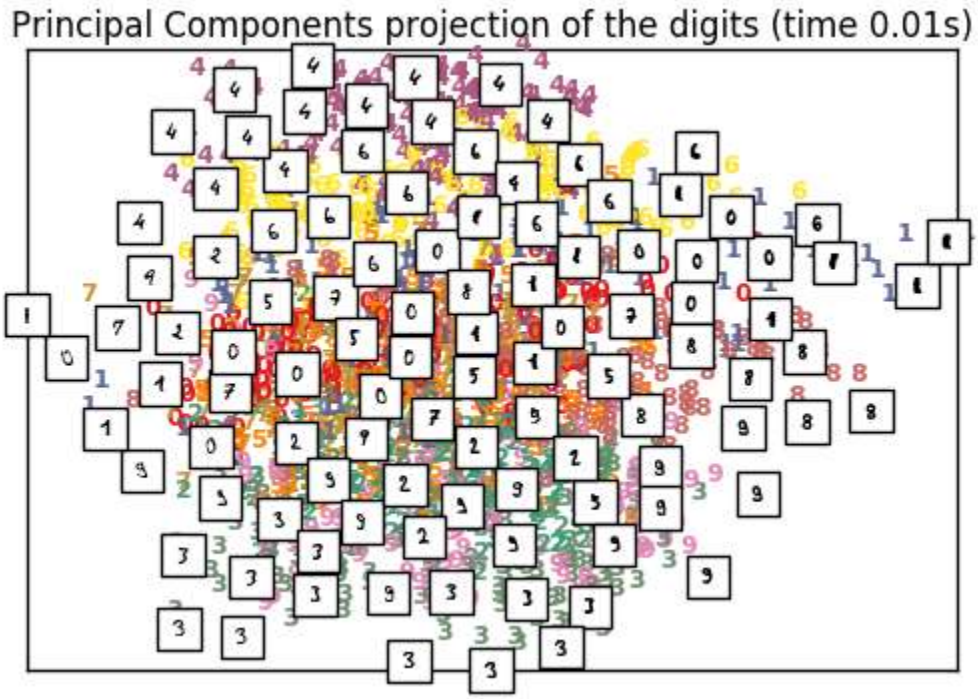
```
def plot_embedding(X, title=None):
    x_min, x_max = np.min(X, 0), np.max(X, 0)
    X = (X - x_min) / (x_max - x_min)
    plt.figure()
    ax = plt.subplot(111)
    for i in range(X.shape[0]):
        plt.text(X[i, 0], X[i, 1], str(digits.target[i]),
                 color=plt.cm.Set1(y[i] / 10.),
                 fontdict={'weight': 'bold', 'size': 9})
    if hasattr(offsetbox, 'AnnotationBbox'):
        ## only print thumbnails with matplotlib > 1.0
        shown_images = np.array([[1., 1.]])  # just something big
        for i in range(digits.data.shape[0]):
            dist = np.sum((X[i] - shown_images) ** 2, 1)
            if np.min(dist) < 4e-3:
                ## don't show points that are too close
                continue
            shown_images = np.r_[shown_images, [X[i]]]
            imagebox = offsetbox.AnnotationBbox(
                offsetbox.OffsetImage(digits.images[i], cmap=plt.cm.gray_r),
                X[i])
            ax.add_artist(imagebox)
    plt.xticks([]), plt.yticks([])
    if title is not None:
        plt.title(title)

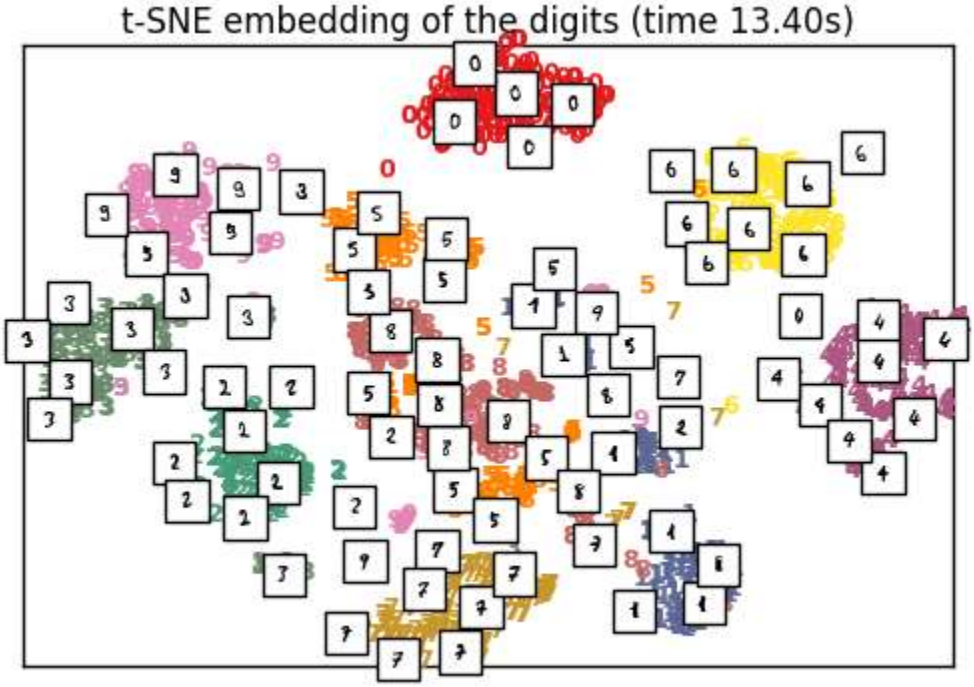
#----------------------------------------------------------------------
## Plot images of the digits
n_img_per_row = 20
img = np.zeros((10 * n_img_per_row, 10 * n_img_per_row))
for i in range(n_img_per_row):
    ix = 10 * i + 1
    for j in range(n_img_per_row):
        iy = 10 * j + 1
        img[ix:ix + 8, iy:iy + 8] = X[i * n_img_per_row + j].reshape((8, 8))
plt.imshow(img, cmap=plt.cm.binary)
plt.xticks([])
plt.yticks([])
plt.title('A selection from the 64-dimensional digits dataset')
## Computing PCA
print("Computing PCA projection")
t0 = time()
X_pca = decomposition.TruncatedSVD(n_components=2).fit_transform(X)
plot_embedding(X_pca,
               "Principal Components projection of the digits (time %.2fs)" %
               (time() - t0))
## Computing t-SNE
print("Computing t-SNE embedding")
tsne = manifold.TSNE(n_components=2, init='pca', random_state=0)
t0 = time()
X_tsne = tsne.fit_transform(X)
plot_embedding(X_tsne,
               "t-SNE embedding of the digits (time %.2fs)" %
               (time() - t0))
plt.show()Copy Code
```

- Implementation Time

Tsne: 13.40 s

PCA: 0.01 s





Where and When to use t-SNE?

How can a Data Scientist use t-SNE?

Well, for the data scientist, the main problem while using t-SNE is the black box-type nature of the algorithm. This impedes the process of providing inferences and insights based on the results. Another problem with the algorithm is that it doesn't always give a similar output on successive runs.

So then, how could you use the algorithm? The best way to use the algorithm is to use it for exploratory data analysis. It will give you a good sense of patterns hidden inside the data. It can also be used as an input parameter for other classification & clustering algorithms.

How can a Machine Learning Hacker use t-SNE?

Reduce the dataset to 2 or 3 dimensions and stack this with a non-linear stacker. Using a holdout set for stacking/blending. Then, you can boost the t SNE vectors using XGboost to get better results.

How can a Data Science Enthusiasts use t-SNE?

For data science enthusiasts beginning to work with data science, this algorithm presents the best opportunities for research and performance enhancements. A few research papers have attempted to improve the algorithm's time complexity by utilizing linear functions, but an optimal solution is still required. Research papers on implementing t-SNE for various NLP problems and image processing applications are unexplored territory and have enough scope. Exploring similar data points,

utilizing nonlinear dimensionality reduction techniques, and supporting independent authors could significantly contribute to this field's advancement.

Common Mistakes While Interpreting

Following are a few common fallacies to avoid while interpreting the results of t-SNE:

1. For the algorithm to execute properly, the perplexity should be smaller than the number of points. Also, the suggested perplexity is in the range of (5 to 50)
2. Sometimes, different runs with the same hyperparameters may produce different results.
3. Cluster sizes in any t SNE plot must not be evaluated for standard deviation, dispersion, or any other similar measures. This is because t-SNE machine learning expands denser clusters and contracts sparser clusters to even out cluster sizes, which is one reason for its crisp and clear plots.
4. Distances between clusters may change because global geometry is closely related to optimal perplexity. In a dataset with many clusters with different numbers of elements, one perplexity cannot optimize distances for all clusters.
5. Patterns may also be found in random noise, so multiple runs of the algorithm with different sets of hyperparameters must be checked before deciding if a pattern exists in the data.
6. Different cluster shapes may be observed at different perplexity levels.

7. Topology cannot be analyzed based on a single t-SNE plot; multiple plots must be observed before being assessed.

Conclusion

PCA, a widely used dimensionality reduction method, simplifies data by projecting it onto a lower-dimensional space while preserving the variance. However, it faces challenges when dealing with nonlinear data. On the other hand, t-SNE, which emphasizes local relationships, is adept at visualizing complex structures. While PCA may struggle with nonlinear techniques, t-SNE shines in scenarios where such methods are prevalent. For instance, in facial expression recognition and text comparison using word vectors, t-SNE offers valuable insights. T-SNE machine learning effectively complements PCA and other dimensionality reduction algorithms by highlighting large pairwise distances and redundant features, providing versatile solutions across various domains.